CCS3241

SOFTWARE TESTING

AND

QUALITY ASSURANCE

STUDY MANUAL

Table of Contents

BACKGROUND AND RATIONALE....4
AIM....4
OBJECTIVES....4
METHOD OF TEACHING....5
ASSESSMENT....5
PART A Introduction to Software Testing....7
Definition of a Bug,....7
Role of a Software Tester....8
Software Development Models....9
Software Testing Axioms....10
Software Testing Terms and Definitions....11
PART B Fundamentals of Software Testing....27
Testing Strategies and Techniques....27
Structural and Functional testing....28
What is Static Testing?....34
Static Black Box Techniques....35
Dynamic Black Box Testing Techniques....36
PART C White Box Testing....37
Static White Box Testing....37
Dynamic White Box Testing....38
PART D Special Types of Testing....40
Configuration Testing....40
Compatibility testing....42
Graphical User Interface Testing....44
Documentation Testing....45
Security Testing....46
PART E Web site Testing....49
How to Test Web Application or Website....49
PART F Testing Tools....53
Benefits of Automation Testing....53
Random Testing....55
Bug Bashes and Beta Testing....57
What is the Objective behind organising a Bug bash?....57
Why should you conduct a Bug bash?....57
Who facilitates the Bug bash?....57
When to conduct a Bug bash?....58

PART G Test Planning.....61
Test Planning.....61
Test Cases.....62
Bug life cycle.....68
PART H Software Quality Assurance.....74
Definition of Quality.....74
Testing and Quality Assurance at Workplace.....75
Test Management and Organizational Structure.....79
Software Quality Assurance Metrics.....81
PART I Organizational Structure.....87
CMM (Capability Maturity Model).....87
ISO 9000.....90
Software Engineering Standards.....92

BACKGROUND AND RATIONALE

This course is designed for the software engineering professional to gain a greater understanding of the key ingredients in creating and/or managing a successful testing program to meet project needs. Topics covered include test lifecycle planning, test design & coverage analysis, complexity, levels of testing such as unit, integration, system, performance and stress testing. Best practice strategies in software testing such as verification & validation, early lifecycle testing, risk based testing and automation will also be examined including exposure to test automation methods and tools.

AIM

The aim of this course is to help student understand the techniques for software testing and quality assurance and develop skills that will enable them to construct software that is of high quality and reliable.

OBJECTIVES

By the end of the course, students should be able to:

1. Explain the common concepts of software testing and quality assurance;
2. Analyse the different types of system test categories;
3. Develop test plans for software projects that are under development;
4. Apply automation testing techniques in system testing;
5. Describe the frameworks and models used to provide software quality; and
6. Analyse the methods employed in software quality assurance.

METHOD OF TEACHING

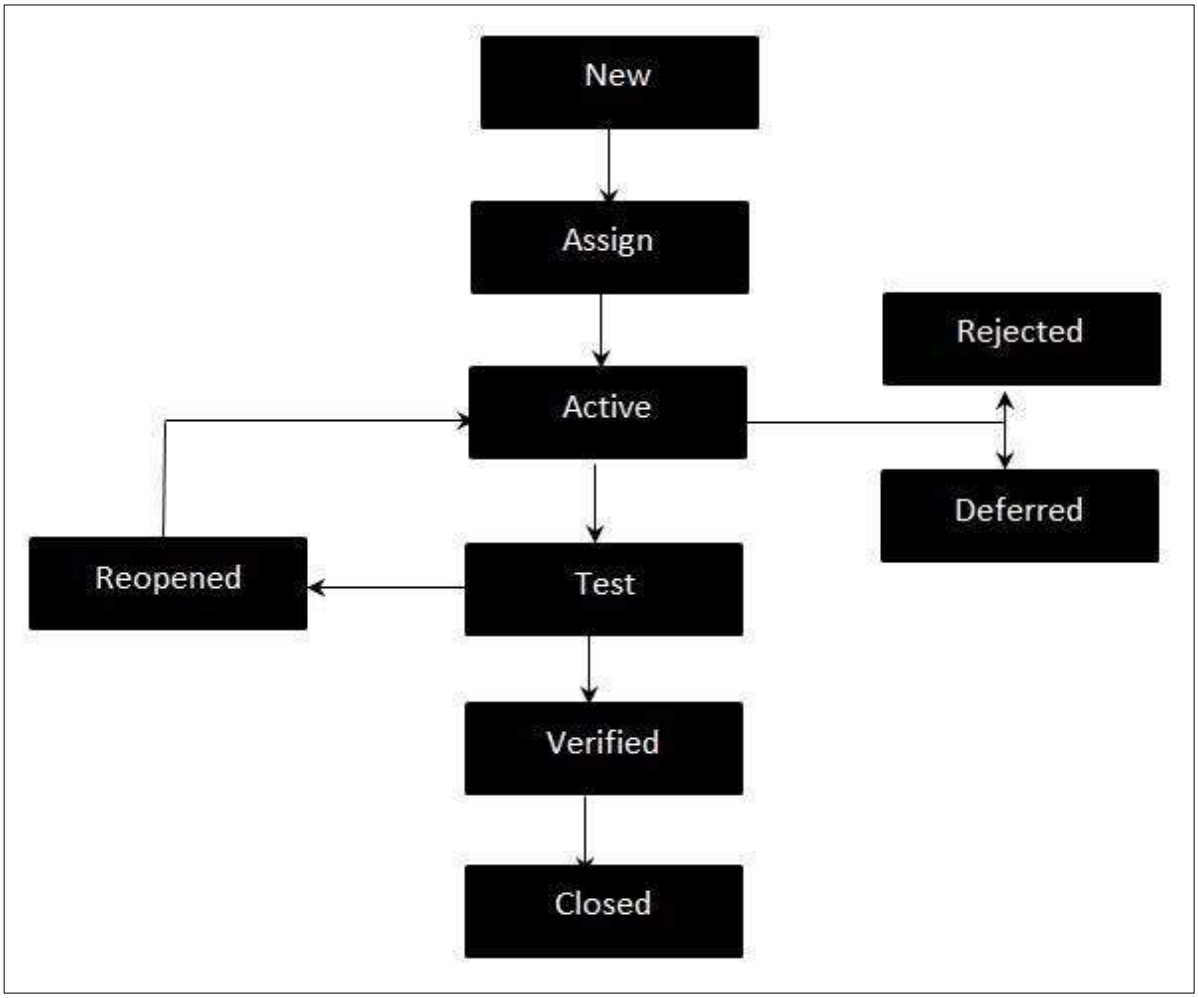
ASSESSMENT		
Continuous assessment		40%
Test	20%	
Practical Exercises	10%	
Practical Assignment		10%
Final Examination		60%
Total		100%

PART A Introduction to Software Testing

Definition of a Bug,

In Software testing, when the expected and actual behavior is not matching, an incident needs to be raised. An incident may be a Bug. It is a programmer's fault where a programmer intended to implement a certain behavior, but the code fails to correctly conform to this behavior because of incorrect implementation in coding. It is also known as Defect.

Following is the workflow of Bug Life Cycle:



Parameters of a Bug:

The Following details should be part of a Bug:

- Date of issue, author, approvals and status.
- Severity and priority of the incident.
- The associated test case that revealed the problem
- Expected and actual results.
- Identification of the test item and environment.
- Description of the incident with steps to Reproduce
- Status of the incident
- Conclusions, recommendations and approvals.

Role of a Software Tester

A Software tester (software test engineer) should be capable of designing test suites and should have the ability to understand usability issues. Such a tester is expected to have sound knowledge of software test design and test execution methodologies. It is very important for a software tester to have great communication skills so that he can interact with the development team efficiently. The roles and responsibilities for a usability software tester are as follows:

1. A Software Tester is responsible for designing testing scenarios for usability testing.
2. He is responsible for conducting the testing, thereafter analyze the results and then submit his observations to the development team.
3. He may have to interact with the clients to better understand the product requirements or in case the design requires any kind of modifications.
4. Software Testers are often responsible for creating test-product documentation and also has to participate in testing related walk through.

A software tester has different sets of roles and responsibilities. He should have in depth knowledge about software testing. He should have a good understanding about the system which means technical (GUI or non-GUI human interactions) as well as functional product aspects. In order to create test cases it is important that the software tester is aware of various testing techniques and which approach is best for a particular system. He should know what are various phases of software

testing and how testing should be carried out in each phase. The responsibilities of the software tester include:

1.Creation of test designs, test processes, test cases and test data.

2.Carry out testing as per the defined procedures.

3.Participate in walkthroughs of testing procedures.

4.Prepare all reports related to software testing carried out.

5.Ensure that all tested related work is carried out as per the defined standards and procedures.

Software Development Models

The **software development models** are the various processes or methodologies that are being selected for the development of the project depending on the project's aims and goals. There are many development life cycle models that have been developed in order to achieve different required objectives. The models specify the various stages of the process and the order in which they are carried out.

The selection of model has very high impact on the testing that is carried out. It will define the what, where and when of our planned testing, influence regression testing and largely determines which test techniques to use.

There are various Software development models or methodologies. They are as follows:

1.Waterfall model

2.V model

3.Incremental model

4.RAD model

5.Agile model

6.Iterative model

7.Spiral model

8.Prototype model

Choosing right model for developing of the software product or application is very important. Based on the model the development and testing processes are carried out.

Different companies based on the software application or product, they select the type of development model whichever suits to their application. But these days in market the '**Agile Methodology**' is the most used model.

'**Waterfall Model**' is the very old model. In 'Waterfall Model' testing starts only after the development is completed. Because of which there are many defects and failures which are reported at the end. So,the cost of fixing these issues are high. Hence, these days people are preferring 'Agile Model'. In 'Agile Model' after every sprint there is a demo-able feature to the customer. Hence customer can see the features whether they are satisfying their need or not.

'**V-model**' is also used by many of the companies in their product. 'V-model' is nothing but 'Verification' and 'Validation' model. In 'V-model' the developer's life cycle and tester's life cycle are mapped to each other. In this model testing is done side by side of the development.

Likewise 'Incremental model', 'RAD model', 'Iterative model' and 'Spiral model' are also used based on the requirement of the customer and need of the product.

Start learning about the models with Waterfall model and its advantages and disadvantages.

Software Testing Axioms

In software testing field, there are several so called 'fundamental truths' which every tester should know and keep during checking mobile or web software products. These statements are verified and supported by constant practice and the rich experience of various testers.

Testing Axioms Are the Following:

- complete testing is impossible.
- quality control is always risk-based.
- absence of bugs cannot be shown with the help of testing.
- One discovered error causes the appearance of many other bugs.

An unexperienced tester may believe that he is able to detect all bugs of the software product. He issure that no errors, any at all, will be missed. But it is impossible to check every system element, every input variable, etc.

Why Is It Impossible to Completely Test a Product?

- The quantity of potential input combinations is huge.
- In accordance, the number of possible outputs is very large too.

➢The amount of available paths throughthe software is huge.

➢The product specification is not objective. A tester may discover an issue, but a stakeholder does not find it to be a real bug.

The whole life is not enough to check all test conditions and develop a perfect product. To check every product aspect may require even the work of whole generation of testers. So a specialist should keep this statement in mind during functional testing or performance testing.

As it is not possible to check all test scenarios, then the untested areas will be certainly left. That is why software testing is a risk-based activity. One cannot know where there may be bugs and where not. If a tester misses a serious bug and discovers it after the release, then the cost of this bug will be very high.

Usually bugs do not appear one by one, sometimes they are interconnected. A tester should be careful when he has discovered one error during black box testing or usability testing, because nearby there may be several more bugs. It can be easily explained.

Why May Bugs Follow Each Other?

➢It was a bad day for a programmer. Thedeveloper’s low spirit or a bad mood influences the quality of work. In such a case, the code will be raw.

➢People are slaves of their habits. The developers may repeat the same mistake again and again.

➢One bug may be only the tip of the iceberg. At first sight, some bugs may seem to be unrelated. But if one examines them closely and deep, he will find out that the whole design or architecture does not match the specification at all.

Software Testing Terms and Definitions

This glossary of testing terms contains a large number of frequently used terms, and is an excerpt of the book *Testing And Quality Assurance Of IT Systems*, by ReQtest founder Ulf Eriksson.

The glossary is meant to help you get familiar with words and phrases commonly used in testing and requirements work. You can use the glossary as the basis for introducing these terms into your organization or standardizing their use.

A

Acceptance testing
The final test level. Conducted by users with the purpose to accept or reject the system before release.

Actual result
The system status or behaviour after you conduct a test. An anomaly or deviation is when your actual results differ from the expected results.

Ad hoc testing
Testing carried out informally without test cases or other written test instructions.

Agile development
A development method that emphasizes working in short iterations. Automated testing is often used. Requirements and solutions evolve through close collaboration between team members that represent both the client and supplier. (Also read: Agile Software Development- 5 Trends to Watch Out For In 2019)

Alpha testing
Operational testing conducted by potential users, customers, or an independent test team at the vendor's site. Alpha testers should not be from the group involved in the development of the system, in order to maintain their objectivity. Alpha testing is sometimes used as acceptance testing by the vendor.

Anomaly
Any condition that deviates from expectations based on requirements specifications, design documents, standards etc. A good way to find anomalies is by testing the software.

B

Beta testing
Test that comes after alpha tests, and is performed by people outside of the organization that built the system. Beta testing is especially valuable for finding usability flaws and configuration problems.

Big-bang integration
An integration testing strategy in which every component of a system is assembled and tested together; contrast with other integration testing strategies in which system components are integrated one at a time.

Black box testing
Testing in which the test object is seen as a "black box" and the tester has no knowledge of its internal structure. The opposite of white box testing.

Bottom-up integration
An integration testing strategy in which you start integrating components from the lowest level of the system architecture. Compare to big-bang integration and top-down integration.

Boundary value analysis
A black box test design technique that tests input or output values that are on the edge of what is allowed or at the smallest incremental distance on either side of an edge. For example, an input field that accepts text between 1 and 10 characters has six boundary values: 0, 1, 2, 9, 10 and 11 characters.

BS 7925-1
A testing standards document containing a glossary of testing terms. BS stands for 'British Standard'.

BS 7925-2
A testing standards document that describes the testing process, primarily focusing on component testing. BS stands for ‘British Standard’.

Bug
A slang term for fault, defect, or error. Originally used to describe actual insects causing malfunctions in mechanical devices that predate computers. The International Software Testing Qualifications Board (ISTQB) glossary explains that “a human being can make an error (mistake), which produces a defect (fault, bug) in the program code, or in a document. If a defect in code is executed, the system may fail to do what it should do (or do something it shouldn’t), causing a failure. Defects in software, systems or documents may result in failures, but not all defects do so.” See also debugging.

C

Capture/playback tool
See record and playback tool.

CAST
A general term for automated testing tools. Acronym for computer-aided software testing.

CCB
See change control board.

Change control board
A group responsible for evaluating, prioritizing, and approving/rejecting requested changes to an IT system.

Change request
A type of document describing a needed or desired change to the system.

Checklist
A simpler form of test case, often merely a document with short test instructions (“one-liners”). An advantage of checklists is that they are easy to develop. A disadvantage is that they are less structured than test cases. Checklists can complement test cases well. In exploratory testing, checklists are often used instead of test cases.

Client
The part of an organization that orders an IT system from the internal IT department or from an external supplier/vendor. See also supplier.

CMMI
Capability Maturity Model Integration. A framework for improving process efficiency in systems development and maintenance.

Code coverage
A generic term for analysis methods that measure the proportion of code in a system that is executed by testing. Expressed as a percentage, for example, 90 % code coverage.

Code review
See Review.

Code standard
Description of how a programming language should be used within an organization. See also naming standard.

Compilation
The activity of translating lines of code written in a human-readable programming language into machine code that can be executed by the computer.

Component
The smallest element of the system, such as class or a DLL.

Component integration testing
Another term for integration test.

Component testing
Test level that evaluates the smallest elements of the system. See also component. Also known as unit test, program test and module test.

Configuration management
Routines for version control of documents and software/program code, as well as managing multiple system release versions.

Configuration testing
A test to confirm that the system works under different configurations of hardware and software, such as testing a website using different browsers.

Context-driven testing
Testing which makes use of debugging techniques inspired by real-world usage conditions. It is a method of testing which encourages testers to develop testing opportunities based on the specific details of any given situation.

COTS
Commercial Off the Shelf. Software that can be bought on the open market. Also called “packaged” software.

D

Daily build
A process in which the test object is compiled every day in order to allow daily testing. While it ensures that defect reports are reported early and regularly, it requires automated testing support.

Debugging
The process in which developers identify, diagnose, and fix errors found. See also bug and defect.

Decision table
A test design and requirements specification technique. A decision table describes the logical conditions and rules for a system. Testers use the table as the basis for creating test cases.

Defect
A flaw in a component or system that can cause the component or system to fail to perform its required function. A defect, if encountered during execution, may cause a failure of the component or system.

Defect report
A document used to report a defect in a component, system, or document. Also known as an incident report.

Deliverable
Any product that must be delivered to someone other than the author of the product. Examples of deliverables are documentation, code and the system.

Desk checking
A static testing technique in which the tester reads code or a specification and “executes” it in his mind.

Document review
See review.

Driver
See test driver.

DSDM
Dynamic Systems Development Method. An iterative development approach.

Dynamic testing
Testing performed while the system is running. Execution of test cases is one example.

E

End-to-end testing
Testing used to test whether the performance of an application from start to finish conforms with the behaviour that is expected from it. This technique can be used to identify system dependencies and confirm the integrity of data transfer across different system components remains.

Entry criteria
Criteria that must be met before you can initiate testing, such as that the test cases and test plans are complete.

Equivalence partitioning
A test design technique based on the fact that data in a system is managed in classes, such as intervals. Because of this, you only need to test a single value in every equivalence class. For example, you can assume that a calculator performs all addition operations in the same way; so if you test one addition operation, you have tested the entire equivalence class.

Error
A human action that produces an incorrect result.

Error description
The section of a defect report where the tester describes the test steps he/she performed, what the

outcome was, what result he/she expected, and any additional information that will assist in troubleshooting.

Error guessing
Experience-based test design technique where the tester develops test cases based on his/her skill and intuition, and experience with similar systems and technologies.

Execute
Run, conduct. When a program is executing, it means that the program is running. When you execute or conduct a test case, you can also say that you are running the test case.

Exhaustive testing
A test approach in which you test all possible inputs and outputs.

Exit criteria
Criteria that must be fulfilled for testing to be considered complete, such as that all high-priority test cases are executed, and that no open high-priority defect remains. Also known as completion criteria.

Expected result
A description of the test object's expected status or behaviour after the test steps are completed. Part of the test case.

Exploratory testing
A test design technique based on the tester's experience; the tester creates the tests while he/she gets to know the system and executes the tests.

External supplier
A supplier/vendor that doesn't belong to the same organization as the client/buyer. See also internal supplier.

Extreme programming
An agile development methodology that emphasizes the importance of pair programming, where two developers write program code together. The methodology also implies frequent deliveries and automated testing.

F

Factory acceptance test
Acceptance testing carried out at the supplier's facility, as opposed to a site acceptance test, which is conducted at the client's site.

Failure
Deviation of the component or system under test from its expected result.

FAT
See Factory Acceptance Test.

Fault Injection
A technique used to improve test coverage by deliberately inserting faults to test different code paths, especially those that handle errors and which would otherwise be impossible to observe.

Formal review
A review that proceeds according to a documented review process that may include, for example, review meetings, formal roles, required preparation steps, and goals. Inspection is an example of a formal review.

Functional integration
An integration testing strategy in which the system is integrated one function at a time. For example, all the components needed for the “search customer” function are put together and tested one by one.

Functional testing
Testing of the system’s functionality and behaviour; the opposite of non-functional testing.

G

Gray-box testing
Testing which uses a combination of white box and black box testing techniques to carry out software debugging on a system whose code the tester has limited knowledge of.

I

IEEE 829
An international standard for test documentation published by the IEEE organization. The full name of the standard is IEEE Standard for Software Test Documentation. It includes templates for the test plan, various test reports, and handover documents.

Impact analysis
Techniques that help assess the impact of a change. Used to determine the choice and extent of regression tests needed.

Incident
A condition that is different from what is expected, such a deviation from requirements or test cases.

Incident report
See defect report.

Independent testing
A type of testing in which testers’ responsibilities are divided up in order to maintain their objectivity. One way to do this is by giving different roles the responsibility for various tests. You can use different sets of test cases to test the system from different points of view.

Informal review
A review that isn’t based on a formal procedure.

Inspection
An example of a formal review technique.

Installation test
A type of test meant to assess whether the system meets the requirements for installation and

uninstallation. This could include verifying that the correct files are copied to the machine and that a shortcut is created in the application menu.

Instrumentation code
Code that makes it possible to monitor information about the system's behaviour during execution. Used when measuring code coverage, for example.

Integration testing
A test level meant to show that the system's components work with one another. The goal is to find problems in interfaces and communication between components.

Internal supplier
Developer that belongs to the same organization as the client. The IT department is usually the internal supplier. See also external supplier.

ISTQB
International Software Testing Qualifications Board. ISTQB is responsible for international programs for testing certification.

Iteration
A development cycle consisting of a number of phases, from formulation of requirements to delivery of part of an IT system. Common phases are analysis, design, development, and testing. The practice of working in iterations is called iterative development.

J

JUnit
A framework for testing Java applications, specifically designed for automated testing of Java components.

L

Load testing
A type of performance testing conducted to evaluate the behaviour of a component or system with increasing load, e.g. numbers of concurrent users and/or numbers of transactions. Used to determine what load can be handled by the component or system. See also performance testing and stress testing.

M

Maintainability
A measure of how easy a given piece of software code is to modify in order to correct defects, improve or add functionality.

Maintenance
Activities for managing a system after it has been released in order to correct defects or to improve or add functionality. Maintenance activities include requirements management, testing, development amongst others.

Module testing
See component testing.

MTBF
Mean time between failures. The average time between failures of a system.

N

Naming standard
The standard for creating names for variables, functions, and other parts of a program. For example, strName, sName and Name are all technically valid names for a variable, but if you don't adhere to one structure as the standard, maintenance will be very difficult.

Negative testing
A type of testing intended to show that the system works well even if it is not used correctly. For example, if a user enters text in a numeric field, the system should not crash.

Non-functional testing
Testing of non-functional aspects of the system, such as usability, reliability, maintainability, and performance.

NUnit
An open source framework for automated testing of components in Microsoft .Net applications.

O

Open source
A form of licensing in which software is offered free of charge. Open source software is frequently available via download from the internet, from www.sourceforge.net for example.

Operational testing
Tests carried out when the system has been installed in the operational environment (or simulated operational environment) and is otherwise ready to go live. Intended to test operational aspects of the system, e.g. recoverability, co-existence with other systems and resource consumption.

Outcome
The result after a test case has been executed.

P

Pair programming
A software development approach where two developers sit together at one computer while programming a new system. While one developer codes, the other makes comments and observations, and acts as a sounding board. The technique has been shown to lead to higher quality thanks to the de facto continuous code review – bugs and errors are avoided because the team catches them as the code is written.

Pair testing
Test approach where two persons, e.g. two testers, a developer and a tester, or an end-user and a

tester, work together to find defects. Typically, they share one computer and trade control of it while testing. One tester can act as observer when the other performs tests.

Performance testing
A test to evaluate whether the system meets performance requirements such as response time or transaction frequency.

Positive testing
A test aimed to show that the test object works correctly in normal situations. For example, a test to show that the process of registering a new customer functions correctly when using valid test data.

Postconditions
Environmental and state conditions that must be fulfilled after a test case or test run has been executed.

Preconditions
Environmental and state conditions that must be fulfilled before the component or system can be tested. May relate to the technical environment or the status of the test object. Also known as prerequisites or preparations.

Prerequisites
See preconditions.

Priority
The level of importance assigned to e.g. a defect.

Professional tester
A person whose sole job is testing.

Program testing
See component testing.

Q

Quality
The degree to which a component, system or process meets specified requirements and/or user/customer needs and expectations.

Quality assurance (QA)
Systematic monitoring and evaluation of various aspects of a component or system to maximize the probability that minimum standards of quality are being attained.

R

Record and playback tool
Test execution tool for recording and playback of test cases often used to support automation of regression testing. Also known as capture/playback.

Regression testing
A test activity generally conducted in conjunction with each new release of the system, in order to

detect defects that were introduced (or discovered) when prior defects were fixed. Compare to Re-testing.

Release
A new version of the system under test. The release can be either an internal release from developers to testers, or release of the system to the client. See also release management.

Release management
A set of activities geared to create new versions of the complete system. Each release is identified by a distinct version number. See also versioning and release.

Release testing
A type of non-exhaustive test performed when the system is installed in a new target environment, using a small set of test cases to validate critical functions without going into depth on any one of them. Also called smoke testing – a funny way to say that, as long as the system does not actually catch on fire and start smoking, it has passed the test.

Requirements management
A set of activities covering gathering, elicitation, documentation, prioritization, quality assurance and management of requirements for an IT system.

Requirements manager
The person responsible for requirements management Also known as Requirements Lead or Business Analyst.

Re-testing
A test to verify that a previously-reported defect has been corrected.

Retrospective meeting
A meeting at the end of a project/a sprint during which the team members evaluate the work and learn lessons that can be applied to the next project or sprint.

Review
A static test technique in which the reviewer reads a text in a structured way in order to find defects and suggest improvements. Reviews may cover requirements documents, test documents, code, and other materials, and can range from informal to formal.

Reviewer
A person involved in the review process that identifies and documents discrepancies in the item being reviewed. Reviewers are selected in order to represent different areas of expertise, stakeholder groups and types of analysis.

Risk
A factor that could result in future negative consequences. Is usually expressed in terms of impact and likelihood.

Risk-based testing
A structured approach in which test cases are chosen based on risks. Test design techniques like boundary value analysis and equivalence partitioning are risk-based. All testing ought to be risk-based.

RUP
The Rational Unified Process; a development methodology from IBM's Rational software division.

S

Sandwich integration
An integration testing strategy in which the system is integrated both top-down and bottom-up simultaneously. Can save time, but is complex.

Scalability testing
A component of non-functional testing, used to measure the capability of software to scale up or down in terms of its non-functional characteristics.

Scenario
A sequence of activities performed in a system, such as logging in, signing up a customer, ordering products, and printing an invoice. You can combine test cases to form a scenario especially at higher test levels.

Scrum
An iterative, incremental framework for project management commonly used with agile software development.

Session-based testing
An approach to testing in which test activities are planned as uninterrupted, quite short, sessions of test design and execution, often used in conjunction with exploratory testing.

Severity
The degree of impact that a defect has on the development or operation of a component or system.

Site acceptance testing (SAT)
Acceptance testing carried out onsite at the client's location, as opposed to the developer's location. Testing at the developer's site is called factory acceptance testing (FAT).

Smoke testing
See release testing.

State transition testing
A test design technique in which a system is viewed as a series of states, valid and invalid transitions between those states, and inputs and events that cause changes in state.

Static testing
Testing performed without running the system. Document review is an example of a static test.

Stress testing
Testing meant to assess how the system reacts to workloads (network, processing, data volume) that exceed the system's specified requirements. Stress testing shows which system resource (e.g. memory or bandwidth) is first to fail.

Structural testing
See white box testing.

Stub
See test stub

Supplier
The organization that supplies an IT system to a client. Can be internal or external. Also called vendor. Contrast with Client.

System
The integrated combination of hardware, software, and documentation.

System integration testing
A test level designed to evaluate whether a system can be successfully integrated with other systems (e.g. that the tested system works well with the finance system). May be included as part of system-level testing, or be conducted as its own test level in between system testing and acceptance testing.

System testing
Test level aimed at testing the complete integrated system. Both functional and non-functional tests are conducted.

T

Test automation
The process of writing programs that perform test steps and verify the result.

Test basis
The documentation on which test cases are based.

Test case
A structured test script that describes how a function or feature should be tested, including test steps, expected results preconditions and postconditions.

Test data
Information that completes the test steps in a test case with e.g. what values to input. In a test case where you add a customer to the system the test data might be customer name and address. Test data might exist in a separate test data file or in a database.

Test driven development
A development approach in which developers writes test cases before writing any code.

Test driver
A software component (driver) used during integration testing in order to emulate (i.e. to stand in for) higher-level components of the architecture. For example, a test driver can emulate the user interface during tests.

Test environment
The technical environment in which the tests are conducted, including hardware, software, and test tools. Documented in the test plan and/or test strategy.

Test execution
The process of running test cases on the test object.

Test level
A group of test activities organized and carried out together in order to meet stated goals. Examples of levels of testing are component, integration, system, and acceptance test.

Test log
A document that describes testing activities in chronological order.

Test manager
The person responsible for planning the test activities at a specific test level. Usually responsible for writing the test plan and test report. Often involved in writing test cases.

Test object
The part or aspects of the system to be tested. Might be a component, subsystem, or the system as a whole.

Test plan
A document describing what should be tested by whom, when, how, and why. The test plan is bounded in time, describing system testing for a particular version of a system, for example. The test plan is to the test leader what the project plan is to the project manager.

Test policy
A document that describes how an organization runs its testing processes at a high level. It may contain a description of test levels according to the chosen life cycle model, roles and responsibilities, required/expected documents, etc.

Test process
The complete set of testing activities, from planning through to completion. The test process is usually described in the test policy.

Test report
A document that summarizes the process and outcome of testing activities at the conclusion of a test period. Contains the test manager’s recommendations, which in turn are based on the degree to which the test activities attained its objectives. Also called test summary report.

Test run
A group of test cases e.g. all the test cases for system testing with owner and end-date.

Tests on one test level are often grouped into a series of tests, i.e. two-week cycles consisting of testing, re-testing, and regression testing. Each series can be a test run.

Test script
Automated test case that the team creates with the help of a test automation tool. Sometimes also used to refer to a manual test case, or to a series of interlinked test cases.

Test specification
A document containing a number of test cases that include steps for preparing and resetting the system. In a larger system you might have one test specification for each subsystem.

Test strategy
Document describing how a system is usually tested.

Test stub
A test program used during integration testing in order to emulate lower-level components. For example, you can replace a database with a test stub that provides a hard-coded answer when it is called.

Test suite
A group of test cases e.g. all the test cases for system testing.

Testing
A set of activities intended to evaluate software and other deliverables to determine if that they meet requirements, to demonstrate that they are fit for purpose and to find defects.

Third-party component
A part of an IT system that is purchased as a packaged/complete product instead of being developed by the supplier/vendor.

Top-down integration
An integration test strategy, in which the team starts to integrate components at the top level of the system architecture.

TPI
Test Process Improvement. A method of measuring and improving the organization's maturity with regard to testing.

Traceability
Analysis of a prior chain of events, as well as the ability to follow an object such as a document or a program through various versions. Traceability enables you to determine the impact of a change in requirements, assuming you also develop a traceability matrix.

Traceability matrix
A table showing the relationship between two or more baselined documents, such as requirements and test cases, or test cases and defect reports. Used to assess what impact a change will have across the documentation and software, for example, which test cases will need to be run when given requirements change.

U

UML
Unified Modeling Language. A technique for describing the system in the form of use cases. See also use case.

Unit test
See component test.

Unit test framework
Software or class libraries that enable developers to write test code in their regular programming language. Used to automate component and integration testing.

Usability
The capability of the software to be understood, learned, used and attractive to the user.

Usability testing
A test technique for evaluating a system's usability. Frequently conducted by users performing tasks in the system while they describe their thought process out loud.

Use case
A type of requirements document in which the requirements are written in the form of sequences that describe how various actors in the system interact with the system.

V

V-model
A software development lifecycle model that describes requirements management, development, and testing on a number of different levels.

Validation
Tests designed to demonstrate that the developers have built the correct system. Contrast with verification, which means testing that the system has been built correctly. A large number of validation activities take place during acceptance testing.

Verification
Tests designed to demonstrate that the developers have built the system correctly. Contrast with validation, which means testing that the correct system has been built. A large number of verification activities take place during component testing.

Versioning
Various methods for uniquely identifying documents and source files, e.g. with a unique version number. Each time the object changes, it should receive a new version number. See also release management.

W

Waterfall model
A sequential development approach consisting of a series of phases carried out one by one. This approach is not recommended due to a number of inherent problems.

White box testing
A type of testing in which the tester has knowledge of the internal structure of the test object. White box testers may familiarize themselves with the system by reading the program code, studying the database model, or going through the technical specifications. Contrast with black box testing.

PART B Fundamentals of Software Testing

Testing Strategies and Techniques

Software Testing is a type of investigation to find out if there is any default or error present in the software so that the errors can be reduced or removed to increase the quality of the software and to check whether it fulfills the specifies requirements or not. According to Glen Myers, software testing has the following objectives:

•The process of investigating and checking a program to find whether there is an error or not and does it fulfill the requirements or not is called testing.

•When the number of errors found during the testing is high, it indicates that the testing was good and is a sign of good test case.

•Finding an unknown error that wasn't discovered yet is a sign of a successful and a good test case.

The main objective of software testing is to design the tests in such a way that it systematically finds different types of errors without taking much time and effort so that less time is required for the development of the software. The overall strategyfor testing software includes:

Before testing starts, it's necessary to identify and specify the requirements of the product in a quantifiable manner. Different characteristics quality of the software is there such as maintainability that means the ability to update and modify, the probability that means to find and estimate any risk, and usability that means how it can easily be used by the customers or end-users. All these characteristic qualities should be specified in a particular order to obtain clear test results without any error.

1.**Specifying the objectives of testing in a clear and detailed manner.** Several objectives of testing are there such as effectiveness that means how effectively the software can achieve the target, any failure that means inability to fulfill the requirements and perform functions, and the cost of defects or errors that mean the cost required to fix the error. All these objectives should be clearly mentioned in the test plan.

2.**For the software, identifying the user's category and developing a profile for each user.** Use cases describe the interactions and communication among different classes of users and the system to achieve the target. So as to identify the actual requirement of the users and then testing the actual use of the product.

3.**Developing a test plan to give value and focus on rapid-cycle testing.** Rapid Cycle Testing is a type of test that improves quality by identifying and measuring the any changes that need to be required for improving the process of software. Therefore, a test plan is an important and effective document that helps the tester to perform rapid cycle testing.

4.**Robust software is developed that is designed to test itself.** The software should be capable of detecting or identifying different classes of errors. Moreover, software design should allow automated and regression testing which tests the software to find out if there is any adverse or side effect on the features of software due to any change in code or program.

5.**Before testing, using effective formal reviews as a filter.** Formal technical reviews is technique to identify the errors that are not discovered yet. The effective technical reviews conducted before testing reduces a significant amount of testing efforts and time duration required for testing software so that the overall development time of software is reduced.

6.**Conduct formal technical reviews to evaluate the nature, quality or ability of the test strategy and test cases.** The formal technical review helps in detecting any unfilled gap in the testing approach. Hence, it is necessary to evaluate the ability and quality of the test strategy and test cases by technical reviewers to improve the quality of software.

7.**For the testing process, developing a approach for the continuous development.** As a part of a statistical process control approach, a test strategy that is already measured should be used for software testing to measure and control the quality during the development of software.

Structural and Functional testing

Structural Testing

In this section, we are going to understand **structural testing**, which is a significant part of **Software testing**.

And we also learn about its needs, **types of structural testing, tools compatible for Structural testing, advantage, and disadvantage.**

Introduction of Structural Testing

Another type of **software testing** technique is **Structural testing**, which is used to test the internal design of the software or structure of the coding for the particular software.

In this testing, the development team members are included in the testing team to execute the software's internal design. The working of structural testing is opposite to **Behavioral testing**.

Types of Structural Testing

Mutation testing

- It is used to check the quality of the test case that should fail the mutant code.
- Mutation testing involves the development of new tests to be implemented on the software for its testing process.
- When we identify various errors, it implies that either the program is correct or the test case is inefficient in locating the fault.

- In the mutation testing, the developers make small modifications to the previously accessible software tests and generate a mutant of the old software test.
- It used to cause an error in the program, which implies that the mutation testing is performed to evaluate the test case's productivity.

Data flow testing

- It is a group of testing approaches used to observe the control flow of programs to discover the sequence of variables as per the series of events.
- It implements a control flow graph and analysis the points where the codes can change the data.
- If we execute the data flow testing technique, the information is kept safe and unchanged during the code's implementation.

Control flow testing

- The **control flow testing** is the basic model of **Structural testing**.
- It is to check the implementation order of commands or statements of the code over a control structure.
- In the control flow testing, a specific part of an extensive program is selected by the test engineer to set the testing path.
- Generally, the control flow testing technique is used in unit testing.
- In this testing, the entire test is based on how the control is executed during the code.
- The complete information of all the software's features and logic is necessary to execute the control flow testing.

Slice-based testing

- It was initially created and established to keep the software.
- The basic idea is to sort the complete code into small chunks and then evaluate each portion carefully.
- The slice-based testing is very beneficial for the maintenance of the software along with fixing the application too.

Structural Testing Tools

Like other testing has their tools, structural testing also contains some open-source and commercial tools that have their functionality.

Some of the most commonly used tools for structural testing are as follows:

- **Cucumber**
- **JBehave**
- **Cfix**
- **JUnit**

Functional Testing

It is a type of software testing which is used to verify the functionality of the software application, whether the function is working according to the requirement specification. In functional testing, each function tested by giving the value, determining the output, and verifying the actual output with the expected value. Functional testing performed as black-box testing which is presented to confirm that the functionality of an application or system behaves as we are expecting. It is done to verify the functionality of the application.

Functional testing also called as black-box testing, because it focuses on application specification rather than actual code. Tester has to test only the program rather than the system.

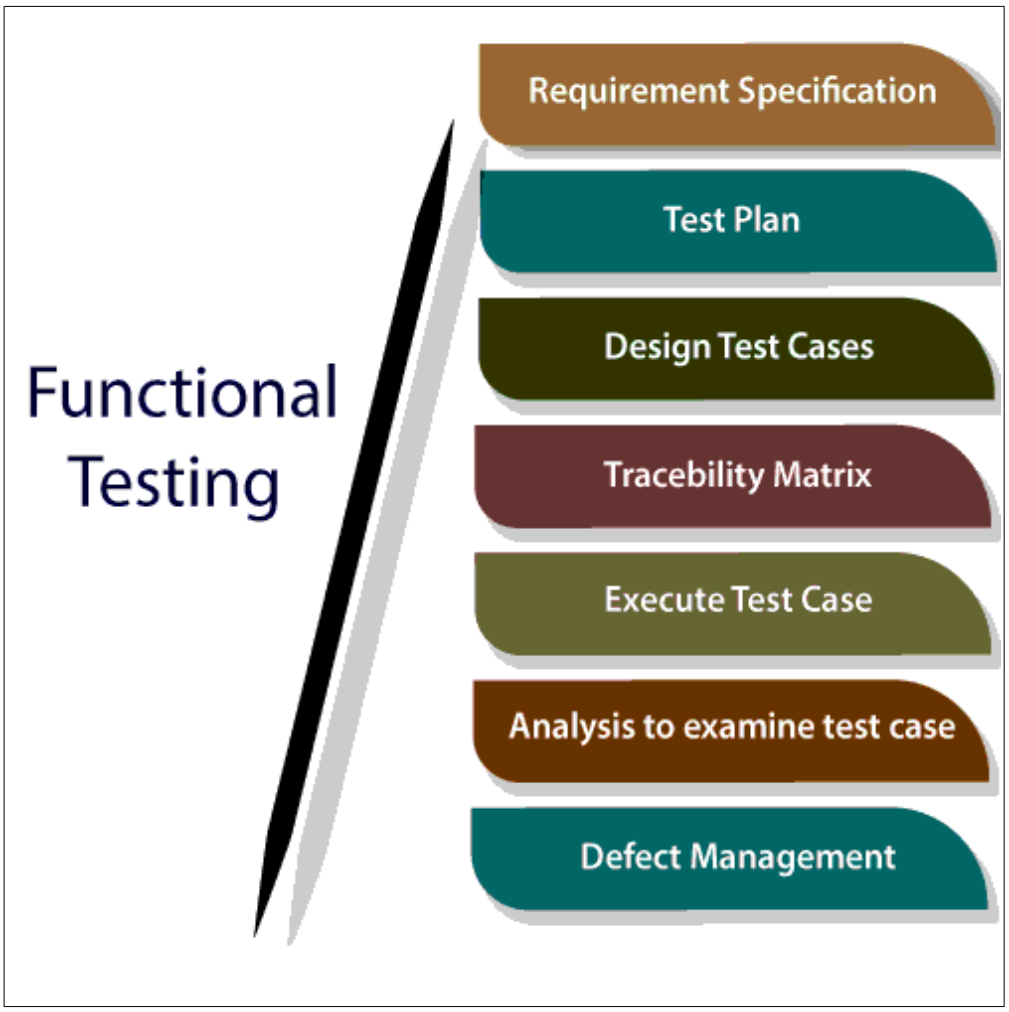
Goal of functional testing

The purpose of the functional testing is to check the primary entry function, necessarily usable function, the flow of screen GUI. Functional testing displays the error message so that the user can easily navigate throughout the application.

What is the process of functional testing?

Testers follow the following steps in the functional testing:

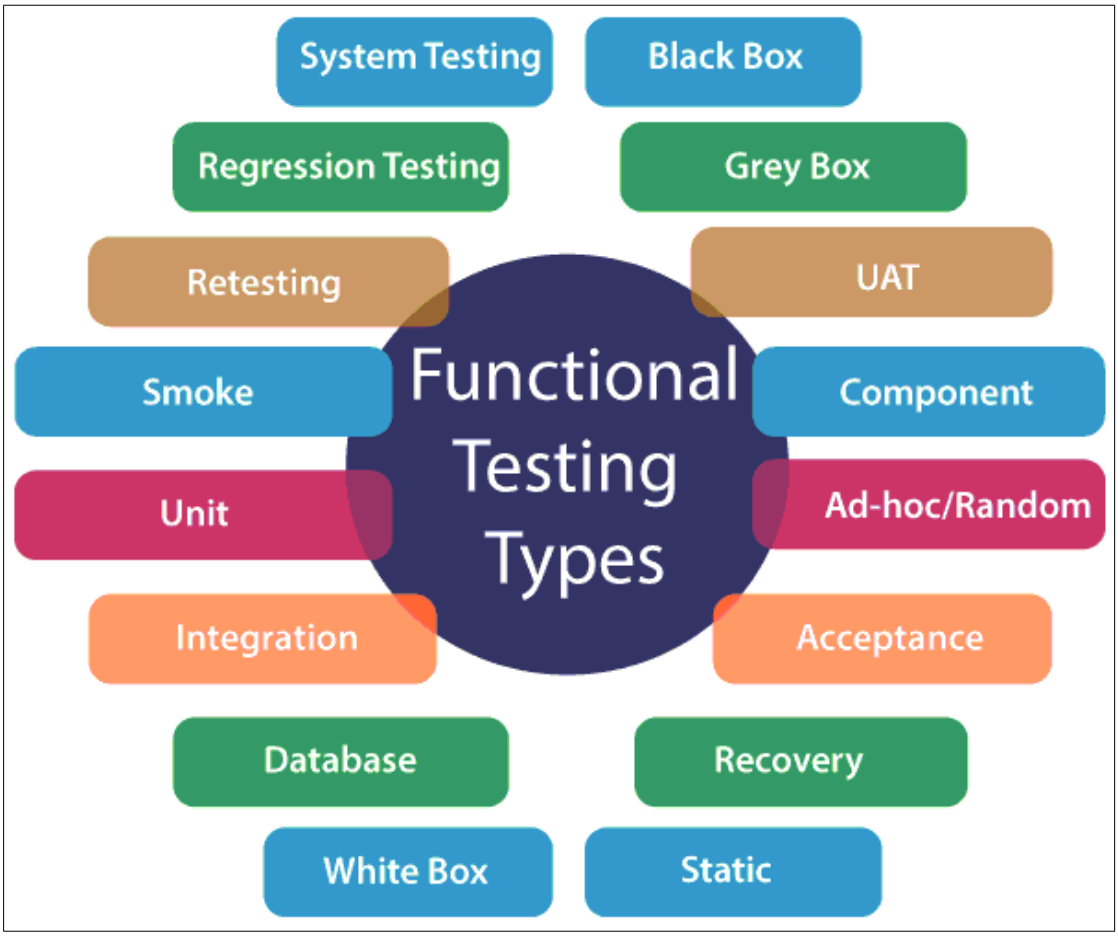
- Tester does verification of the requirement specification in the software application.
- After analysis, the requirement specification tester will make a plan.
- After planning the tests, the tester will design the test case.
- After designing the test, case tester will make a document of the traceability matrix.
- The tester will execute the test case design.
- Analysis of the coverage to examine the covered testing area of the application.
- Defect management should do to manage defect resolving



The main objective of functional testing is checking the functionality of the software system. It concentrates on:

- **Basic Usability:** Functional Testing involves the usability testing of the system. It checks whether a user can navigate freely without any difficulty through screens.
- **Accessibility:** Functional testing test the accessibility of the function.
- **Mainline function:** It focuses on testing the main feature.
- **Error Condition:** Functional testing is used to check the error condition. It checks whether the error message displayed.

Types of functional testing.



Unit Testing: Unit testing is a type of software testing, where the individual unit or component of the software tested. Unit testing, examine the different part of the application, by unit testing functional testing also done, because unit testing ensures each module is working correctly.

The developer does unit testing. Unit testing is done in the development phase of the application.

Smoke Testing: Functional testing by smoke testing. Smoke testing includes only the basic (feature) functionality of the system. Smoke testing is known as "***Build Verification Testing***." Smoke testing aims to ensure that the most important function work.

For example, Smoke testing verifies that the application launches successfully will check that GUI is responsive.

Sanity Testing: Sanity testing involves the entire high-level business scenario is working correctly. Sanity testing is done to check the functionality/bugs fixed. Sanity testing is little advance than smoke testing.

For example, login is working fine; all the buttons are working correctly; after clicking on the button navigation of the page is done or not.

Regression Testing: This type of testing concentrate to make sure that the code changes should not side effect the existing functionality of the system. Regression testing specifies when bug arises in the system after fixing the bug, regression testing concentrate on that all parts are working or not. Regression testing focuses on is there any impact on the system.

Integration Testing: Integration testing combined individual units and tested as a group. The purpose of this testing is to expose the faults in the interaction between the integrated units.

Developers and testers perform integration testing.

White box testing: White box testing is known as Clear Box testing, code-based testing, structural testing, extensive testing, and glass box testing, transparent box testing. It is a software testing method in which the internal structure/design/ implementation tested known to the tester.

The white box testing needs the analysis of the internal structure of the component or system.

Black box testing: It is also known as behavioral testing. In this testing, the internal structure/ design/ implementation not known to the tester. This type of testing is functional testing. Why we called this type of testing is black-box testing, in this testing tester, can't see the internal code.

For example, A tester without the knowledge of the internal structures of a website tests the web pages by using the web browser providing input and verifying the output against the expected outcome.

User acceptance testing: It is a type of testing performed by the client to certify the system according to requirement. The final phase of testing is user acceptance testing before releasing the software to the market or production environment. UAT is a kind of black-box testing where two or more end-users will involve.

Retesting: Retesting is a type of testing performed to check the test cases that were unsuccessful in the final execution are successfully pass after the defects fixed. Usually, tester assigns the bug when they find it while testing the product or its component. The bug allocated to a developer, and he fixes it. After fixing, the bug is assigned to a tester for its verification. This testing is known as retesting.

Database Testing: Database testing is a type of testing which checks the schema, tables, triggers, etc. of the database under test. Database testing may involve creating complex queries to load/stress test the database and check its responsiveness. It checks the data integrity and consistency.

Example: let us consider a banking application whereby a user makes a transaction. Now from database testing following, things are important. They are:

- Application store the transaction information in the application database and displays them correctly to the user.
- No information lost in this process
- The application does not keep partially performed or aborted operation information.
- The user information is not allowed individuals to access by the

Ad-hoc testing: Ad-hoc testing is an informal testing type whose aim is to break the system. This type of software testing is unplanned activity. It does not follow any test design to create the test cases. Ad-hoc testing is done randomly on any part of the application; it does not support any structured way of testing.

Recovery Testing: Recovery testing is used to define how well an application can recover from crashes, hardware failure, and other problems. The purpose of recovery testing is to verify the system's ability to recover from testing points of failure.

Static Testing: Static testing is a software testing technique by which we can check the defects in software without actually executing it. Static testing is done to avoid errors in the early stage of the development as it is easier to find failure in the early stages. Static testing used to detect the mistakes that may not found in dynamic testing.

What is Static Testing?

Static Testing is a type of software testing in which software application is tested without code execution. Manual or automated reviews of code, requirement documents and document design are done in order to find the errors. The main objective of static testing is to improve the quality of software applications by finding errors in early stages of software development process.

Static testing involves manual or automated reviews of the documents. This review is done during an initial phase of testing to catch Defect early in STLC. It examines work documents and provides review comments. It is also called Non-execution testing or verification testing.

Examples of Work documents-

- Requirement specifications
- Design document
- Source Code
- Test Plans
- Test Cases
- Test Scripts
- Help or User document
- Web Page content

What is Dynamic Testing?

Under **Dynamic Testing**, a code is executed. It checks for functional behavior of software system, memory/cpu usage and overall performance of the system. Hence the name “Dynamic”

The main objective of this testing is to confirm that the software product works in conformance with the business requirements. This testing is also called an Execution technique or validation testing.

Dynamic testing executes the software and validates the output with the expected outcome. Dynamic testing is performed at all levels of testing and it can be either black or white box testing

Static Black Box Techniques

Equivalence Partitioning

Testers can divide possible inputs into groups or “partitions”, and test only one example input from each group. For example, if a system requires a user’s birth date and provides the same response for all users under the age of 18, and a different response for users over 18, it is sufficient for testers to check one birth date in the “under 18” group and one date in the “over 18” group.

Boundary Value Analysis

Testers can identify that a system has a special response around a specific boundary value. For example, a specific field may accept only values between 0 and 99. Testers can focus on the boundary values (-1, 0, 99 and 100), to see if the system is accepting and rejecting inputs correctly.

Decision Table Testing

Many systems provide outputs based on a set of conditions. Testers can then identify “rules” which are a combination of conditions, identify the outcome of each rule, and design a test case for each rule.

For example, a health insurance company may provide different premium based on the age of the insured person (under 40 or over 40) and whether they are a smoker or not. This generates a decision table with four rules and up to four outcomes—below is an example with three possible outcomes.

State Transition Testing

In some systems, significant responses are generated when the system transitions from one state to another. A common example is a login mechanism which allows users to authenticate, but after a specific number of login attempts, transition to a different state, locking the account.

If testers identify a state transition mechanism, they can design test cases that probe the system when it transitions states. For example, for a system that locks the account after five failed login attempts, a test case can check what happens at the sixth login attempt.

Error Guessing

This technique involves testing for common mistakes developers make when building similar systems. For example, testers can check if the developer handled null values in a field, text in a numeric field or numbers in a text-only field, and sanitization of inputs—whether it is possible to submit user inputs that contain executable code, which has security significance.

A specific type of error guessing is testing for known software vulnerabilities that can affect the system under test.

Dynamic Black Box Testing Techniques

Following are the prominent Test Strategy amongst the many used in Black box Testing

- **Equivalence Class Testing:** It is used to minimize the number of possible test cases to an optimum level while maintains reasonable test coverage.
- **Boundary Value Testing:** Boundary value testing is focused on the values at boundaries. This technique determines whether a certain range of values are acceptable by the system or not. It is very useful in reducing the number of test cases. It is most suitable for the systems where an input is within certain ranges.
- **Decision Table Testing**: A decision table puts causes and their effects in a matrix. There is a unique combination in each column.

PART C White Box Testing

Static White Box Testing

Static white-box testing is the process of carefully and methodically reviewing the software design, architecture, or code for bugs without executing it. It's sometimes referred to as structural analysis.

The obvious reason to perform static white-box testing is to find bugs early and to find bugs that would be difficult to uncover or isolate with dynamic black-box testing. Having a team of testers concentrate their efforts on the design of the software at this early stage of development is highly cost effective.

A side benefit of performing static white-box testing is that it gives the team's black-box testers ideas for test cases to apply when they receive the software for testing. They may not necessarily understand the details of the code, but by listening to the review comments they can identify feature areas that sound troublesome or bug-prone.

Development teams vary in who has the responsibility for static white-box testing. In some teams the programmers are the ones who organize and run the reviews, inviting the software testers as independent observers. In other teams the software testers are the ones who perform this task, asking the programmer who wrote the code and a couple of his peers to assist in the reviews. Ultimately, either approach can work. It's up to the development team to choose what works best for them.

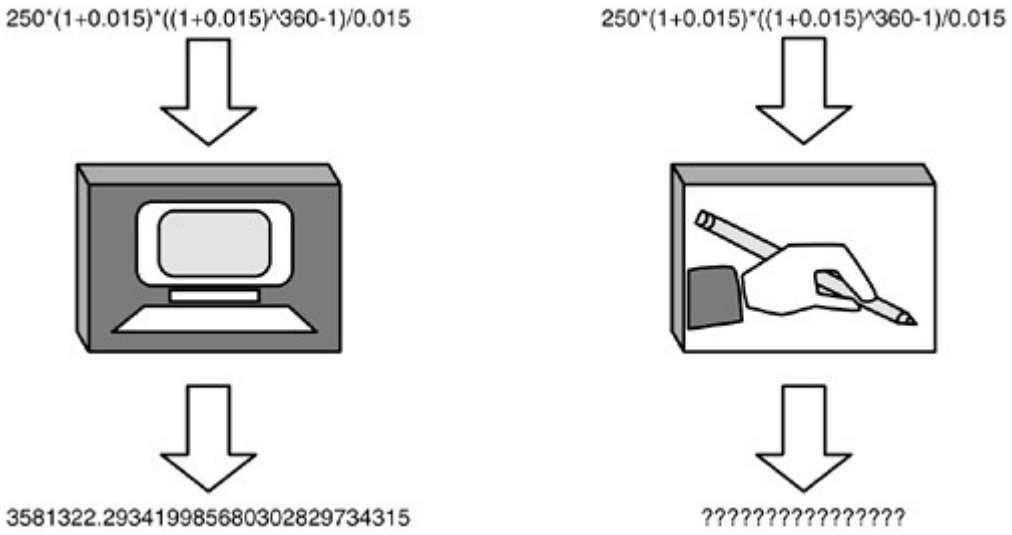
The unfortunate thing about static white-box testing is that it's not always done. Many teams have the misconception that it's too time-consuming, too costly, or not productive. All of these are untruecompared to the alternative of testing, finding, and even not finding bugs at the back end of the project. The problem lies in the perception that a programmer's job is to write lines of code and that any task that takes away from his efficiency of churning out those lines is slowing down the process.

Fortunately, the tide is changing. Many companies are realizing the benefits of testing early and are hiring and training their programmers and testers to perform white-box testing. It's not rocket science (unless you're designing rockets), but getting started requires knowing a few basic techniques. If you're interested in taking it further, the opportunities are huge.

Dynamic White Box Testing

Dynamic white-box testing, in a nutshell, is using information you gain from seeing what the code does and how it works to determine what to test, what not to test, and how to approach the testing. Another name commonly used for dynamic white-box testing is structural testing because you can see and use the underlying structure of the code to design and run your tests.

Why would it be beneficial for you to know what's happening inside the box, to understand how the software works? Consider Figure below. This figure shows two boxes that perform the basic calculator operations of addition, subtraction, multiplication, and division.



If you didn't know how the boxes worked, you would apply the dynamic black-box testing techniques. "Testing the Software with Blinders On." But, if you could look in the boxes and see that one contained a computer and the other contained a person with a pencil and paper, you would probably choose a completely different test approach for each one. Of course, this example is very simplistic, but it makes the point that knowing how the software operates will influence how you test.

Dynamic white-box testing isn't limited just to seeing what the code does. It also can involve directly testing and controlling the software. The four areas that dynamic white-box testing encompasses are

•Directly testing low-level functions, procedures, subroutines, or libraries. In Microsoft Windows, these are called Application Programming Interfaces (APIs).

•Testing the software at the top level, as a completed program, but adjusting your test cases based on what you know about the software's operation.

•Gaining access to read variables and state information from the software to help you determine whether your tests are doing what you thought. And, being able to force the software to do things that would be difficult if you tested it normally.

•Measuring how much of the code and specifically what code you "hit" when you run your tests and then adjusting your tests to remove redundant test cases and add missing ones.

Each area is discussed in the remainder of this chapter. Think about them as you read on and consider how they might be used to test software that you're familiar with.

PART D Special Types of Testing

Configuration Testing

Configuration Testing is the type of Software Testing which verifies the performance of the system under development against various combinations of software and hardware to find out the best configuration under which the system can work without any flaws or issues while matching its functional requirements.

Configuration Testing is the process of testing the system under each configuration of the supported software and hardware. Here, the different configurations of hardware and software means the multiple operating system versions, various browsers, various supported drivers, distinct memory sizes, different hard drive types, various types of CPU etc.

Various Configurations:

•Operating System Configuration:

```
Win XP, Win 7 32/64 bit, Win 8 32/64 bit, Win 10 etc.
```

•Database Configuration:

```
Oracle, DB2, MySql, MSSQL Server, Sybase etc.
```

•Browser Configuration:

```
IE 8, IE 9, FF 16.0, Chrome, Microsoft Edge etc.
```

Objectives of Configuration Testing:

The objective of configuration testing is:

To determine whether the software application fulfills the configurability requirements.

•To identify the defects that were not efficiently found during different testing processes.

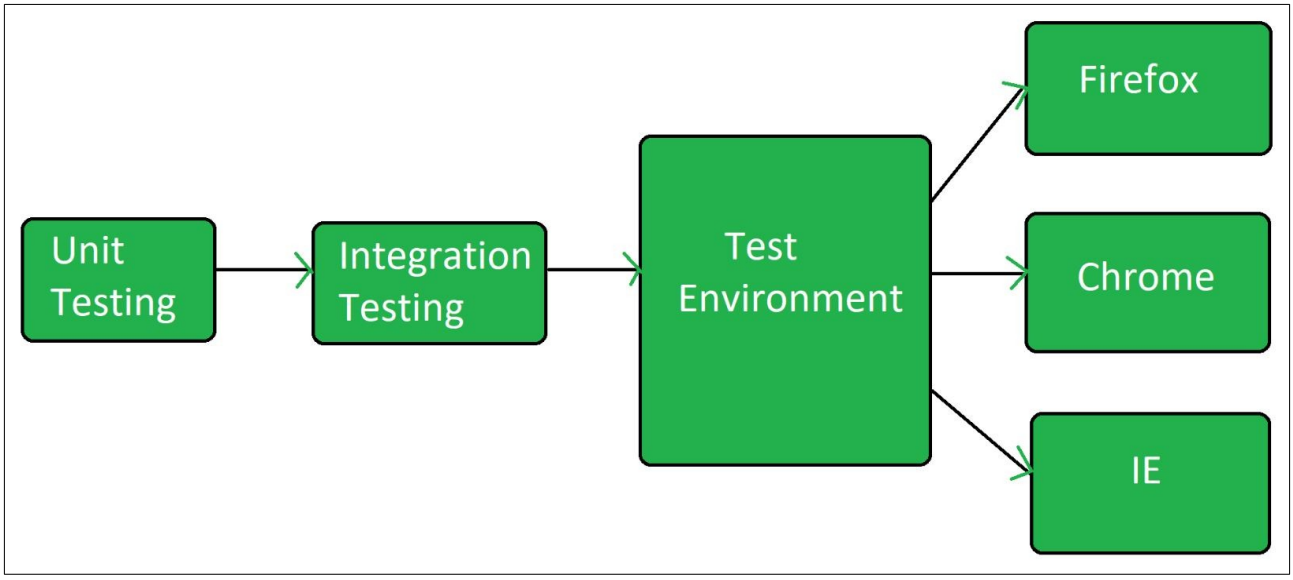
•To determine an optimal configuration of the application under test.

•To do analyse of the performance of software application by changing the hardware and software resources.

•To do analyse of the system efficiency based on the prioritization.

•To verify the degree of ease to how the bugs are reproducible irrespective of the configuration changes.

Configuration Testing Process:



Types of Configuration Testing:
Configuration testing is of 2 types:

1.**Software Configuration Testing:**

Software configuration testing is done over the Application Under Test with various operating system versions and various browser versions etc. It is a time consuming testing as it takes long time to install and uninstall the various software which are to be used for testing. When the build is released, software configuration begins after passing through the unit test and integration test.

Hardware Configuration Testing:
Hardware configuration testing is typically performed in labs where physical machines are used with various hardware connected to them.

When a build is released, the software is installed in all the physical machines to which the hardware is attached and the test is carried out on each and every machine to confirm that the application is working fine. While doing hardware configuration test, the kind of hardware to be tested is spelled out and there are several computer hardware and peripherals which make it next to impossible to execute all the tests.

Configuration Testing can also be classified into following 2 types:

1.**Client level Testing:**
Client level testing is associated with the usability and functionality testing. This testing is done from the point of view of its direct interest of the users.

2.**Server level Testing:**
Server level testing is carried out to determine the communication between the software and the external environment when it is planned to be integrated after the release.

Compatibility testing

Compatibility testing is software testing which comes under the non functional testing category, and it is performed on an application to check its compatibility (running capability) on different platform/environments. This testing is done only when the application becomes stable. Means simply this compatibility test aims to check the developed software application functionality on various software, hardware platforms, network and browser etc. This compatibility testing is very important in product production and implementation point of view as it is performed to avoid future issues regarding compatibility.

Types of Compatibility Testing :
Several examples of compatibility testing are given below.

1. Software :

Testing the compatibility of an application with an Operating System like Linux, Mac, Windows

•Testing compatibility on Database like Oracle SQL server, MongoDB server.

•Testing compatibility on different devices like in mobile phones, computers.

Types based on Version Testing :

There are two types of compatibility testing based on version testing

1.**Forward compatibility testing :** When the behavior and compatibility of a software or hardware is checked with its newer version then it is called as forward compatibility testing.

2.**Backward compatibility testing :** When the behavior and compatibility of a software or hardware is checked with its older version then it is called as backward compatibility testing.

2. Hardware :

Checking compatibility with a particular size of

•RAM

•ROM

•Hard Disk

•Memory Cards

•Processor

•Graphics Card

3. Smartphones :

Checking compatibility with different mobile platforms like android, iOS etc.

4.Network :

Checking compatibility with different :

•Bandwidth

•Operating speed

•Capacity

Along with this there are other types of compatibility testing are also performed such as browser compatibility to check software compatibility with different browsers like Google Chrome, Internet Explorer etc. device compatibility, version of the software and others.

How to perform Compatibility testing ?

Testing the application in a same environment but having different versions. For example, to test compatibility of Facebook application in your android mobile. First check for the compatibility with Android 9.0 and then with Android 10.0 for the same version of Facebook App.

Testing the application in a same versions but having different environment. For example, to test compatibility of Facebook application in your android mobile. First check for the compatibility with a Facebook application of lower version with a Android 10.0(or your choice) and then with a Facebook application of higher version with a same version of Android.

Why compatibility testing is important ?

1.It ensures complete customer satisfaction.

2.It provides service across multiple platforms.

3.Identifying bugs during development process.

Compatibility testing defects :

1.Variety of user interface.

2.Changes with respect to font size.

3.Alignment issues.

4.Issues related to existence of broken frames.

5.Issues related to overlapping of content.

Graphical User Interface Testing

Graphical User Interface Testing (GUI) Testing is the process for ensuring proper functionality of the graphical user interface (GUI) for a specific application.GUI testing generally evaluates a design of elements such as layout, colors and also fonts, font sizes, labels, text boxes, text formatting, captions, buttons, lists, icons, links, and content. GUI testing processes may be either manual or automatic and are often performed by third-party companies, rather than developers or end users.

Feature of Graphical User Interface Testing (GUI):
There are some feature of GUI which are given below:

•It provides a customizable test report.

•It is run tests in parallel or distributed on a Selenium Grid with built-in Selenium Webdriver.

•It allows you to test the functionality from a user's perspective.

•Sometimes the internal functions of the system work correctly but the user interface doesn't then GUI testing is good to have in addition to the other types.

•It provides reliable object identification, even for web elements with dynamic IDs.

Types of Graphical User Interface Testing (GUI) Testing:
There are two types of GUI testing which are given below: Analog Recording, and Object based recording. These are explained as following below.

1.**Analog Recording:**
This is always what people associate with GUI testing tools by analog recording, the testing tool basically captures specific mouse clicks, keyboard presses, and other user actions and then simply stores them in a file for playback. For example, it might record that a user left-clicked at position X = 500 pixels, Y = 400 pixels or typed the word "Search" in a box and pressed the [ENTER] key on their keyboard.

2.**Object based Recording:**
In object based recording, the testing tool is able to connect programmatically to the application being tested and "see" each of the individual user interface components (a button, a text box, a hyperlink) as separate entities and is able to perform operations (click, enter text) and read the state (is it enabled, what is the label text, what is the current value) reliably regardless of where that object is on the screen.

Challenges with Graphical User Interface Testing (GUI) Testing:
There are some challenges that occur during Graphical user interface testing. These are given below.

1.Technology Support

2.Stability of Objects

3.Instrumentation

Documentation Testing

Testing documentation is the documentation of artifacts that are created during or before the testing of a software application. Documentation reflects the importance of processes for the customer, individual and organization.

Projects which contain all documents have a high level of maturity. Careful documentation can save the time, efforts and wealth of the organization.

Types of test document

In software testing, we have various types of test document, which are as follows:



Benefits of using Documentation

- Documentation clarifies the quality of methods and objectives.
- It ensures internal coordination when a customer uses software application.
- It ensures clarity about the stability of tasks and performance.
- It provides feedback on preventive tasks.
- It provides feedback for your planning cycle.
- It creates objective evidence for the performance of the quality management system.
- If we write the test document, we can't forget the values which we put in the first phase.
- It is also a time-saving process because we can easily refer to the text document.
- It is also consistent because we will test on the same value.

The drawback of the test document

- It is a bit tedious because we have to maintain the modification provided by the customer and parallel change in the document.
- If the test documentation is not proper, it will replicate the quality of the application.
- Sometimes it is written by that person who does not have the product knowledge.
- Sometimes the cost of the document will be exceeding its value.

Security Testing

Security Testing s a type of Software Testing that uncovers vulnerabilities, threats, risks in a software application and prevents malicious attacks from intruders. The purpose of Security Tests is to identify all possible loopholes and weaknesses of the software system which might result in a loss of information, revenue, repute at the hands of the employees or outsiders of the Organization.

Why Security Testing is Important?

The main goal of **Security Testing** is to identify the threats in the system and measure its potential vulnerabilities, so the threats can be encountered and the system does not stop functioning or can not be exploited. It also helps in detecting all possible security risks in the system and helps developers to fix the problems through coding.

Types of Security Testing:

There are seven main types of security testing as per Open Source Security Testing methodology manual. They are explained as follows:

Vulnerability Scanning: This is done through automated software to scan a system against known vulnerability signatures.

- **Security Scanning:** It involves identifying network and system weaknesses, and later provides solutions for reducing these risks. This scanning can be performed for both Manual and Automated scanning.
- **Penetration testing**: This kind of testing simulates an attack from a malicious hacker. This testing involves analysis of a particular system to check for potential vulnerabilities to an external hacking attempt.
- **Risk Assessment:** This testing involves analysis of security risks observed in the organization. Risks are classified as Low, Medium and High. This testing recommends controls and measures to reduce the risk.
- **Security Auditing:** This is an internal inspection of Applications and Operating systems for security flaws. An audit can also be done via line by line inspection of code
- **Ethical hacking:** It's hacking an Organization Software systems. Unlike malicious hackers, who steal for their own gains, the intent is to expose security flaws in the system.
- **Posture Assessment:** This combines Security scanning, Ethical Hacking and Risk Assessments to show an overall security posture of an organization.

Example Test Scenarios for Security Testing:

Sample Test scenarios to give you a glimpse of security test cases –

- A password should be in encrypted format
- Application or System should not allow invalid users
- Check cookies and session time for application
- For financial sites, the Browser back button should not work.

Security Testing Tool

Intuitive and easy to use, Acunetix by Invicti helps small to medium-sized organizations ensure their web applications are secure from costly data breaches. It does so by detecting a wide range of web security issues and helping security and development professionals act fast to resolve them

2) Intruder

Intruder is a powerful, automated penetration testing tool that discovers security weaknesses across your IT environment. Offering industry-leading security checks, continuous monitoring and an easy-to-use platform, Intruder keeps businesses of all sizes safe from hackers.

3) Owasp

The Open Web Application Security Project (OWASP) is a worldwide non-profit organization focused on improving the security of software. The project has multiple tools to pen test various software environments and protocols. Flagship tools of the project include

1.Zed Attack Proxy (ZAP – an integrated penetration testing tool)

2.OWASP Dependency Check (it scans for project dependencies and checks against know vulnerabilities)

3.OWASP Web Testing Environment Project (collection of security tools and documentation)

4) WireShark

Wireshark is a network analysis tool previously known as Ethereal. It captures packet in real time and display them in human readable format. Basically, it is a network packet analyzer- which provides the minute details about your network protocols, decryption, packet information, etc. It is an open source and can be used on Linux, Windows, OS X, Solaris, NetBSD, FreeBSD and many other systems. The information that is retrieved via this tool can be viewed through a GUI or the TTY mode TShark Utility.

5) W3af

w3af is a web application attack and audit framework. It has three types of plugins; discovery, audit and attack that communicate with each other for any vulnerabilities in site, for example a discovery plugin in w3af looks for different url’s to test for vulnerabilities and forward it to the audit plugin which then uses these URL’s to search for vulnerabilities.

PART E Web site Testing

Web Testing, or website testing is checking your web application or website for potential bugs before its made live and is accessible to general public. Web Testing checks for functionality, usability, security, compatibility, performance of the web application or website.

During this stage issues such as that of web application security, the functioning of the site, its access to handicapped as well as regular users and its ability to handle traffic is checked

How to Test Web Application or Website

In Software Engineering, the following testing types/technique may be performed depending on your web testing requirements.

1. Functionality Testing of a Website

Functionality Testing of a Website is a process that includes several testing parameters like user interface, APIs, database testing, security testing, client and server testing and basic website functionalities. Functional testing is very convenient and it allows users to perform both manual and automated testing. It is performed to test the functionalities of each feature on the website.

Web based Testing Activities includes:

Test all **links** in your webpages are working correctly and make sure there are no broken links. Links to be checked will include –

Outgoing links

- Internal links
- Anchor Links
- MailTo Links

Test Forms are working as expected. This will include-

- Scripting checks on the form are working as expected. For example- if a user does not fill a mandatory field in a form an error message is shown.
- Check default values are being populated
- Once submitted, the data in the forms is submitted to a live database or is linked to a working email address
- Forms are optimally formatted for better readability

Test Cookies are working as expected. Cookies are small files used by websites to primarily remember active user sessions so you do not need to log in every time you visit a website. Cookie Testing will include

- Testing cookies (sessions) are deleted either when cache is cleared or when they reach their expiry.
- Delete cookies (sessions) and test that login credentials are asked for when you next visit the site.
- **Test HTML and CSS** to ensure that search engines can crawl your site easily. This will include
- Checking for Syntax Errors
- Readable Color Schemas
- Standard Compliance. Ensure standards such W3C, OASIS, IETF, ISO, ECMA, or WS-I are followed.

Test business workflow– This will include

- Testing your end – to – end workflow/ business scenarios which takes the user through a series of webpages to complete.
- Test negative scenarios as well, such that when a user executes an unexpected step, appropriate error message or help is shown in your web application.

Tools that can be used: QTP , IBM Rational , Selenium

2 Usability testing

Usability Testing has now become a vital part of any web based project. It can be **carried out by testers** like you **or a small focus group** similar to the target audience of the web application.

Test the site **Navigation**:

- Menus, buttons or Links to different pages on your site should be easily visible and consistent on all webpages

Test the **Content**:

- Content should be legible with no spelling or grammatical errors.
- Images if present should contain an “alt” text

Tools that can be used: Chalkmark, Contentsquare, and Clixpy

3.Interface Testing:

Three areas to be tested here are – Application, Web and Database Server

- **Application:** Test requests are sent correctly to the Database and output at the client side is displayed correctly. Errors if any must be caught by the application and must be only shown to the administrator and not the end user.
- **Web Server**: Test Web server is handling all application requests without any service denial.
- **Database Server:** Make sure queries sent to the database give expected results.

Test system response when **connection between the three layers** (Application, Web and Database) **cannot be established** and appropriate message is shown to the end user.

Tools that can be used: Ranorex

4. Database Testing:

Database is one critical component of your web application and stress must be laid to test it thoroughly. Testing activities will include-

- Test if any errors are shown while executing queries
- Data Integrity is maintained while creating, updating or deleting data in database.
- Check response time of queries and fine tune them if necessary.
- Test data retrieved from your database is shown accurately in your web application

Tools that can be used: QTP, Selenium

5. Compatibility testing.

Compatibility tests ensures that your web application displays correctly across different devices. This would include-

Browser Compatibility Test: Same website in different browsers will display differently. You need to test if your web application is being displayed correctly across browsers, JavaScript, AJAX and authentication is working fine. You may also check for Mobile Browser Compatibility.

The rendering of web elements like buttons, text fields etc. changes with change in **Operating System**. Make sure your website works fine for various combination of Operating systems such as Windows, Linux, Mac and Browsers such as Firefox, Internet Explorer, Safari etc.

Tools that can be used: Dynatrace

6. Performance Testing:

This will ensure your site works under all loads. Software Testing activities will include but not limited to –

- Website application response times at different connection speeds

- Load test your web application to determine its behavior under normal and peak loads
- Stress test your web site to determine its break point when pushed to beyond normal loads at peak time.
- Test if a crash occurs due to peak load, how does the site recover from such an event
- Make sure optimization techniques like gzip compression, browser and server side cache enabled to reduce load times

Tools that can be used: Loadrunner, JMeter

7. Security testing:

Security Testing is vital for e-commerce website that store sensitive customer information like credit cards. Testing Activities will include-

- Test unauthorized access to secure pages should not be permitted
- Restricted files should not be downloadable without appropriate access
- Check sessions are automatically killed after prolonged user inactivity
- On use of SSL certificates, website should re-direct to encrypted SSL pages.

Intruder

Intruder is a powerful vulnerability scanner that will help you uncover the many weaknesses lurking in your web applications and underlying infrastructure. Offering industry-leading security checks, continuous monitoring and an easy-to-use platform, Intruder keeps businesses of all sizes safe from hackers.

8. Crowd Testing:

You will select a large number of people (crowd) to execute tests which otherwise would have been executed a select group of people in the company. Crowdsourced testing is an interesting and upcoming concept and helps unravel many a unnoticed defects.

Tools that can be used: Crowd Testing Platform

This concludes the tutorial. It includes almost all testing types applicable to your web application.

As a Web-tester its important to note that web testing is quite an arduous process and you are bound to come across many obstacles. One of the major problems you will face is of course **deadline pressure**. Everything is always needed yesterday! The number of times the **code will need changing** is also taxing. Make sure you **plan your work** and know clearly what is expected of you. Its best **define all the tasks** involved in your web testing and then **create a work chart for accurate estimates and planning**.

PART F Testing Tools

Benefits of Automation Testing

Automation testing uses some specific tools to execute the test scripts without any human interference. In automation testing, the test automation engineer writes the test scripts or uses the automation testing tools to execute the application. Whereas, in manual testing, the test engineer write test cases and implement the software based on written test cases. It is the most acceptable way to enhance the efficiency, productivity, and test coverage of Software testing.

The advantages of automation testing are given as follows -

Saves time

Automating the testing process helps the testing team to use less time to validate newly created features. For instance, in manual testing, there is a need to write thousand test cases for a calculator application, but automation makes the process much faster.

Productivity improvement

As during execution, automation tests do not require human intervention, so testing an application can be done late at night, and we can get the results next morning. Software developers and testers require less time on automation testing.

Accuracy improvement

In manual testing, there is a chance of mistakes whether you are an experienced testing engineer. The chances of errors may increase when testing a complex use case. But Automation testing reduces the chances of errors. There is good accuracy, as we will get the same result each time on performing the same test cases.

Test suite reusability

We can reuse the test scripts in automation testing, and we don't need to write the new test scripts again and again. These test cases can be used in various ways, as they are reusable. Reusability helps to reduce the cost and also eliminate the chances of human error.

Ability to test on various platforms

Automation testing allows the user to test the application on different web browsers and operating systems.

Running tests 24/7

In automation testing, we can start the testing process from anywhere in the world and anytime we want. It can also be done remotely if we don't have many approaches or the option to purchase them.

Early bug detection

By automation testing, it is easy to detect critical bugs in the initial phases of software development. It reduces the cost and helps us to spend fewer working hours to fix such problems. It increases the efficiency of the team.

Less human resources

Automation testing requires fewer people to perform a tedious manual test. To implement the automation test script, we need a test automation engineer who can write the test scripts to automate our test

Reduce the expenses

Automation testing is less expensive, as once the test scripts have been built, we can reuse them at any time without any extra cost. While manual testing is more expensive than automation, with manual monitoring, it is typical to execute experiments repeatedly.

Scalability of test cases

In manual testing, we require the involvement of the number of people and number of hours to scale up a project. Whereas the scalability of automation testing is higher, we need adding of test executors to the testing framework.

Consistency

Compared to manual testing, automation testing is more consistent and way faster than executing the regular monotonous tests that cannot be missed but may cause faults when tested manually.

Fast development and delivery

Automated tests can be executed repeatedly and completed rapidly. We do not have to wait for weeks to execute the tests; few hours are enough for execution. Switching from manual to automation reduces the waiting time and boosts development.

Random Testing

Random testing is software testing in which the system is tested with the help of generating random and independent inputs and test cases. Random testing is also named monkey testing. It is a black box assessment outline technique in which the tests are being chosen randomly and the results are being compared by some software identification to check whether the output is correct or incorrect.

Some important points about Random Testing:

1.Random testing was first examined by Melvin Breuer in the year 1971.

2.This testing was initially assessed by Pratima and Vishwani Agrawal in the year 1975 to check the successful output of the software.

3.For random testing, there is also a book that contains some formulas for the number of tests that can be taken and the number of successful results and failure results.

Working Random Testing:

Step-1: Identify Input domain

Step-2: Select test inputs independently/randomly from the input domain

Step-3: Test the system on these inputs and form a random test set

Step-4: Compare the result with system specification

Step-5: If the Report fails then take necessary action.

The below image represents the working of Random Testing more clearly.

Types of Random Testing

1. Random input sequence generation: It is also known as Random Number Generator (RNG) in which a random sequential number or symbols is being generated which cannot be assumed during the random selection.

2. Random sequence of data inputs: In this, all the data are selected randomly for the inputs which can be used during the testing.

3. Random data selection from an existing database: The record where all the data are available from that record only the data can be selected for testing afterward no additional data cannot be added which are not available in the record.

Characteristics of Random Testing:

1.Random testing is implemented when the bug in an application is not recognized.

2.It is used to check the system’s execution and dependability.

3.It saves our time and does not need any extra effort.

4.Random testing is less costly, it doesn’t need extra knowledge for testing the program.

Methods to Implement Random Testing:

To implement the random testing basically, four steps are applied:

1.The user input domain is analyzed.

2.After that, from that domain, the data of test inputs are chosen separately.

3.With the help of these test inputs, the test is executed successfully. These input tests conduct random sets of tests.

4.The outcomes are compared with the system identification. The outcome of the test becomes unsuccessful if any test input doesn’t match with the original one otherwise the outcomes are always successful.

Advantages of Random Testing

1.It is very cheap so that anyone can use this software.

2.It doesn’t need any special intelligence to access the program during the tests.

3.Errors can be traced very easily; it can easily detect the bug throughout the testing.

4.This software is lacking bias means it makes the groups evenly for the testing and it prefers not to repeatedly check the errors as there can be some changes in the codes throughout the testing process.

Disadvantages of Random Testing

1.This software only finds changes errors.

2.They are not practical. Some tests will be of no use for a longer time.

3.Most of the time is consumed by analyzing all the tests.

4.New tests cannot be formed if their data is not available during testing.

Tools used for Random Testing

1.**QuickCheck:** It is a famous test tool, which was introduced for Haskell which is available in many different languages. This tool generates random orders for API calls that are related to the model and the properties of the system which can give successful results after every test.

2.**Randoop:** This tool provides an order of methods and constructor acknowledgment for the classes during the test and generates JUnit tests.

3.**Simulant:** It is a Clojure tool that runs according to the system’s different specifications, the behavior of the model.

4.**Gram Test:** This random testing tool is based on grammar which is written in Java, It utilizes BNF notation for specifying the grammar which is used as input during testing.

Bug Bashes and Beta Testing.

What is a Bug bash?

In software development, a bug bash is a procedure where all the developers, testers, program managers, usability researchers, designers, documentation folks, and even sometimes marketing people, put aside their regular day-to-day duties and “pound on the product” — that is, each exercises the product in every way they can think of. Because each person will use the product in slightly different (or very different) ways, and the product is getting a great deal of use in a short amount of time, this approach may reveal bugs relatively quickly

What is the Objective behind organising a Bug bash?

As the name and definition already mentions, it is bug bash, so the major objective is to find bugs hidden in the application and resolve them before it reaches the end users of the product. However, The Developers, Business Analyst, Project Managers and the Quality Analysts should all be on the same page and there should not be any blame game once bug is found as this is a collective team effort to make the application more stable and robust by providing faster feedback, so as to release a quality product in the market.

Why should you conduct a Bug bash?

•Most importantly, Bug bashes are a relatively cheaper way to find a large number of bugs in a short span.

•Bug bashes provide an opportunity for everyone in the organization to learn about the different parts of the product they are less familiar with.

•It improves cross-team collaborations, communication, and relationships.

•Helps testing the product with wide variety and combination of devices/browsers/mobile versions/mobile OS’s which in general is very difficult to test in a short span.

•People with different experiences in the team can collaborate and also with different perspective product can be tested effectively.

Who facilitates the Bug bash?

Ideally, it is the Tester or the Test Lead who should facililate this.

When to conduct a Bug bash?

It should be conducted before a major release or if there is any crtitical release which may impact the overall working of the product, Bug bashes are advised. The time to schedule may vary according to the collective decision made by the team. Normally, it should be conducted before a week of release or even sooner. A point to note here is that all the cards/tickets that are tagged for the release should be QA Done or either Dev Done before the bug bash is scheduled. It doesnt make any sense to have a bug bash if the feature that is going to be released is half baked.

What is beta testing?

Beta testing is a type of user acceptance testing where the product team gives a nearly finished product to a group of target users to evaluate product performance in the real world.

There is no standard for what a beta test should look like and how to set up beta testing. The actual testing procedure should be relevant to your testing goals. However, there are a few requirements that a product needs to comply with in order to be ready for beta testing:

- The product should be in the “Feature complete” state (it should have all the features that are planned for the release version).
- The product should be stable (test participants should not face unpredictable crashes).
- Test participants should belong to the product’s target audience.
- Test participants should complete real-world scenarios while interacting with an app, and they should do it in the real environment (not in lab environments).

Alpha testing vs. beta testing: what is the difference?

Before discussing different types and approaches to beta testing, it’s essential to differentiate it from alpha testing.

Alpha testing is the testing phase that precedes the beta test phase. Unlike the beta version, the alpha version is usually less stable and might have a limited feature set. Alpha testing is done by an in-house team of developers, designers, and QA specialists. Alpha testers usually mix black-box (testing method in which the internal structure, design, implementation of the item, product, and feature being tested is not known to the tester) and white-box testing (testing method in which the internal structure, design, and implementation of the item being tested is known to the tester) techniques to discover bugs and crashes. Alpha testing can be controlled activity (meaning, it’s possible to reach test participants and ask specific questions about their experience).

Beta testing is usually black-box testing, meaning test participants don’t know anything about the backend and don’t have access to source code. Since beta testing happens most of the time at the end user’s side, it cannot be a controlled activity. Beta testing can provide extremely valuable insights – genuine scenarios of interactions with a product.

For example, during beta testing, it’s possible to match the expected user journey with the actual user journey, by gathering behavioral habits (i.e. whether the user prefers to use mouse or keyboard during the interaction with a software) and emotional responses on interactions (i.e. how your design decisions make your users feel). It’s also possible to create an empathy map based on the actual data from your users.

The role of A/B testing in product design: phases of testing

In the modern software development processes, beta testing should be an integral part of the product development cycle. Ultimately, the moment for conducting a beta test in a development cycle depends on the nature of a product you’re developing. Some teams reserve beta testing only for major releases, while other teams conduct beta testing for both minor and major releases.

Types of beta testing

Below are a few common types of beta testing. Note that beta testing can include several different beta tests of different types.

Closed beta testing vs. open beta testing

Closed beta version (also known as private beta) released to selected people who test its features. Usually, the number of test participants is limited. For example, if you plan to ship a product on the market in the next few months and have a landing page that allows visitors to leave their emails in order to get the latest updates about the product, you can choose beta testers from people who signed up for product updates. Closed beta tests are better suited for beta tests with a limited scope (only core features of the future product). And since the number of unique issues found by each tester decreases as the number of testers increase, it makes sense to limit the total number of test participants.

What you need to run beta testing

- Define the goals in advance
- Recruit the right test participant
- Define the length of the testing period
- Write product documentation
- Share the information about known issues with test participants
- Share results of beta testing with the product team and stakeholders
- Create a clear procedure for collecting feedback
- Use tools for beta testing

PART G Test Planning

Test Planning

A test plan is a detailed document which describes software testing areas and activities. It outlines the test strategy, objectives, test schedule, required resources (human resources, software, and hardware), test estimation and test deliverables.

The test plan is a base of every software's testing. It is the most crucial activity which ensures availability of all the lists of planned activities in an appropriate sequence.

The test plan is a template for conducting software testing activities as a defined process that is fully monitored and controlled by the testing manager. The test plan is prepared by the Test Lead (60%), Test Manager(20%), and by the test engineer(20%).

Types of Test Plan

There are three types of the test plan

Master Test Plan

- Phase Test Plan
- Testing Type Specific Test Plans

Master Test Plan

Master Test Plan is a type of test plan that has multiple levels of testing. It includes a complete test strategy.

Phase Test Plan

A phase test plan is a type of test plan that addresses any one phase of the testing strategy. For example, a list of tools, a list of test cases, etc.

Specific Test Plans

Specific test plan designed for major types of testing like security testing, load testing, performance testing, etc. In other words, a specific test plan designed for non-functional testing.

How to write a Test Plan

Making a test plan is the most crucial task of the test management process. According to IEEE 829, follow the following seven steps to prepare a test plan.

- First, analyze product structure and architecture.
- Now design the test strategy.

- Define all the test objectives.
- Define the testing area.
- Define all the useable resources.
- Schedule all activities in an appropriate manner.
- Determine all the Test Deliverables.

Test plan components or attributes

The test plan consists of various parts, which help us to derive the entire testing activity.

Objectives: It consists of information about modules, features, test data etc., which indicate the aim of the application means the application behavior, goal, etc.

Scope: It contains information that needs to be tested with respective of an application. The Scope can be further divided into two parts:

- In scope
- Out scope

In scope: These are the modules that need to be tested rigorously (in-detail).

Out scope: These are the modules, which need not be tested rigorously.

For example, Suppose we have a Gmail application to test, where **features to be tested** such as **Compose mail, Sent Items, Inbox, Drafts** and the **features which not be tested** such as **Help**, and so on which means that in the planning stage, we will decide that which functionality has to be checked or not based on the time limit given in the product.

Now **how we decide which features not to be tested?**

We have the following aspects where we can decide which feature not to be tested:

1. As we see above that **Help** features is not going to be tested, as it is written and developed by the technical writer and reviewed by another professional writer.
2. Let us assume that we have one application that have **P, Q, R, and S** features, which need to be developed based on the requirements. But here, the S feature has already been designed and used by some other company. So the development team will purchase S from that company and integrate with additional features such as P, Q, and R.

Test Cases

What is a Test Case?

A **Test Case** is a set of actions executed to verify a particular feature or functionality of your software application. A Test Case contains test steps, test data, precondition, postcondition developed for specific test scenario to verify any requirement. The test case includes specific variables or conditions, using which a testing engineer can compare expected and actual results to determine whether a software product is functioning as per the requirements of the customer.

Test Scenario Vs Test Case

Test scenarios are rather vague and cover a wide range of possibilities. Testing is all about being very specific.

For a Test Scenario: Check Login Functionality there many possible test cases are:

- Test Case 1: Check results on entering valid User Id & Password
- Test Case 2: Check results on entering Invalid User ID & Password
- Test Case 3: Check response when a User ID is Empty & Login Button is pressed, and many more

This is nothing but a Test Case

The format of Standard Test Cases

Below is a format of a standard login Test cases example.

Test Case ID	Test Case Description	Test Steps	Test Data	Expected Results	Actual Results	Pass/ Fail
TU01	Check Customer Login with valid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	Userid = guru99 Password = pass99	User should Login into an application	As Expected	Pass
TU02	Check Customer Login with invalid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	Userid = guru99 Password = glass99	User should not Login into an application	As Expected	Pass

This entire table may be created in Word, Excel or any other Test management tool. That's all to Test Case Design

Step 1) A simple test case to explain the scenario would be

Test Case #	Test Case Description
1	Check response when valid email and password is entered

Step 2) Test the Data.

In order to execute the test case, you would need Test Data. Adding it below

Test Case #	Test Case Description	Test Data
1	Check response when valid email and password is entered	Email: guru99@email.com Password: lNf9^Oti7^2h

Identifying test data can be time-consuming and may sometimes require creating test data afresh. The reason it needs to be documented.

Step 3) Perform actions.

In order to execute a test case, a tester needs to perform a specific set of actions on the AUT. This is documented as below:

Test Case #	Test Case Description	Test Steps	Test Data
1	Check response when valid email and password is entered	1) Enter Email Address	Email: guru99@email.com
		2) Enter Password	Password: lNf9^Oti7^2h
		3) Click Sign in	

Many times the Test Steps are not simple as above, hence they need documentation. Also, the author of the test case may leave the organization or go on a vacation or is sick and off duty or is very busy with other critical tasks. A recently hire may be asked to execute the test case. Documented steps will help him and also facilitate reviews by other stakeholders.

Step 4) Check behavior of the AUT.

The goal of test cases in software testing is to check behavior of the AUT for an expected result. This needs to be documented as below

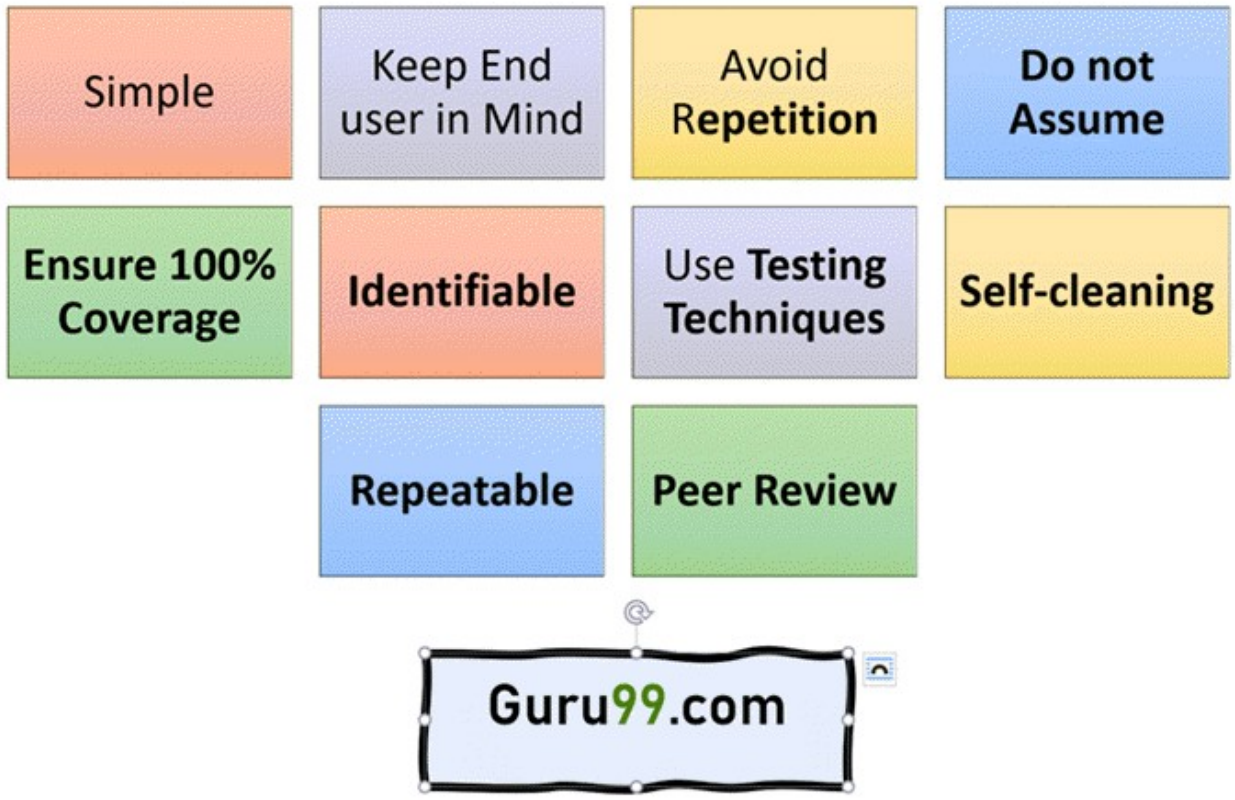
Test Case #	Test Case Description	Test Data	Expected Result
1	Check response when valid email and password is entered	Email: guru99@email.com Password: lNf9^Oti7^2h	Login should be successful

During test execution time, the tester will check expected results against actual results and assign a pass or fail status

Test	Test Case Description	Test Data	Expected	Actual Result	Pass/

Case #			Result		Fail
1	Check response when valid email and password is entered	Email: guru99@email.com Password: lNf9^Oti7^2h	Login should be successful	Login was successful	Pass

Step 5) That apart your test case -may have a field like,
Pre – Condition which specifies things that must be in place before the test can run. For our test case, a pre-condition would be to have a browser installed to have access to the site under test. A test case may also include Post – Conditions which specifies anything that applies after the test case completes. For our test case, a postcondition would be time & date of login is stored in the database



1. Test Cases need to be simple and transparent:

Create test cases that are as simple as possible. They must be clear and concise as the author of the test case may not execute them.

Use assertive language like go to the home page, enter data, click on this and so on. This makes the understanding the test steps easy and tests execution faster.

2. Create Test Case with End User in Mind

The ultimate goal of any software project is to create test cases that meet customer requirements and is easy to use and operate. A tester must create test cases keeping in mind the end user perspective

3. Avoid test case repetition.

Do not repeat test cases. If a test case is needed for executing some other test case, call the test case by its test case id in the pre-condition column

4. Do not Assume

Do not assume functionality and features of your software application while preparing test case. Stick to the Specification Documents.

5. Ensure 100% Coverage

Make sure you write test cases to check all software requirements mentioned in the specification document. Use Traceability Matrix to ensure no functions/conditions is left untested.

6. Test Cases must be identifiable.

Name the test case id such that they are identified easily while tracking defects or identifying a software requirement at a later stage.

7. Implement Testing Techniques

It's not possible to check every possible condition in your software application. Software Testing techniques help you select a few test cases with the maximum possibility of finding a defect.

- **Boundary Value Analysis (BVA):** As the name suggests it's the technique that defines the testing of boundaries for a specified range of values.
- **Equivalence Partition (EP):** This technique partitions the range into equal parts/groups that tend to have the same behavior.
- **State Transition Technique**: This method is used when software behavior changes from one state to another following particular action.
- **Error Guessing Technique:** This is guessing/anticipating the error that may arise while doing manual testing. This is not a formal method and takes advantages of a tester's experience with the application

8. Self-cleaning

The test case you create must return the Test Environment to the pre-test state and should not render the test environment unusable. This is especially true for configuration testing.

9. Repeatable and self-standing

The test case should generate the same results every time no matter who tests it

10. Peer Review.

After creating test cases, get them reviewed by your colleagues. Your peers can uncover defects in your test case design, which you may easily miss.

While drafting a test case to include the following information

- The description of what requirement is being tested
- The explanation of how the system will be tested
- The test setup like a version of an application under test, software, data files, operating system, hardware, security access, physical or logical date, time of day, prerequisites such as other tests and any other setup information pertinent to the requirements being tested
- Inputs and outputs or actions and expected results
- Any proofs or attachments
- Use active case language
- Test Case should not be more than 15 steps
- An automated test script is commented with inputs, purpose and expected results
- The setup offers an alternative to pre-requisite tests
- With other tests, it should be an incorrect business scenario order

Test Case Management Tools

Test management tools are the automation tools that help to manage and maintain the Test Cases. Main Features of a test case management tool are

1.**For documenting Test Cases:** With tools, you can expedite Test Case creation with use of templates

2.**Execute the Test Case and Record the results:** Test Case can be executed through the tools and results obtained can be easily recorded.

3.**Automate the Defect Tracking:** Failed tests are automatically linked to the bug tracker, which in turn can be assigned to the developers and can be tracked by email notifications.

4.**Traceability:** Requirements, Test cases, Execution of Test cases are all interlinked through the tools, and each case can be traced to each other to check test coverage.

5.**Protecting Test Cases:** Test cases should be reusable and should be protected from being lost or corrupted due to poor version control. Test Case Management Tools offer features like

- Naming and numbering conventions

- Versioning
- Read-only storage
- Controlled access
- Off-site backup

Popular Test Management tools are: Quality Center and JIRA

Test Case Template

●Please note that the template used will vary from project to project. Read this tutorial to Learn Test Case Template with Explanation of Important Fields

Bug life cycle

What is Defect/Bug Life Cycle?

Defect Life Cycle or Bug Life Cycle in software testing is the specific set of states that defect or bug goes through in its entire life. The purpose of Defect life cycle is to easily coordinate and communicate current status of defect which changes to various assignees and make the defect fixing process systematic and efficient.

Defect Status

Defect Status or Bug Status in defect life cycle is the present state from which the defect or a bug is currently undergoing. The goal of defect status is to precisely convey the current state or progress of a defect or bug in order to better track and understand the actual progress of the defect life cycle.

Defect States Workflow

The number of states that a defect goes through varies from project to project. Below lifecycle diagram, covers all possible states

●**New:** When a new defect is logged and posted for the first time. It is assigned a status as NEW.

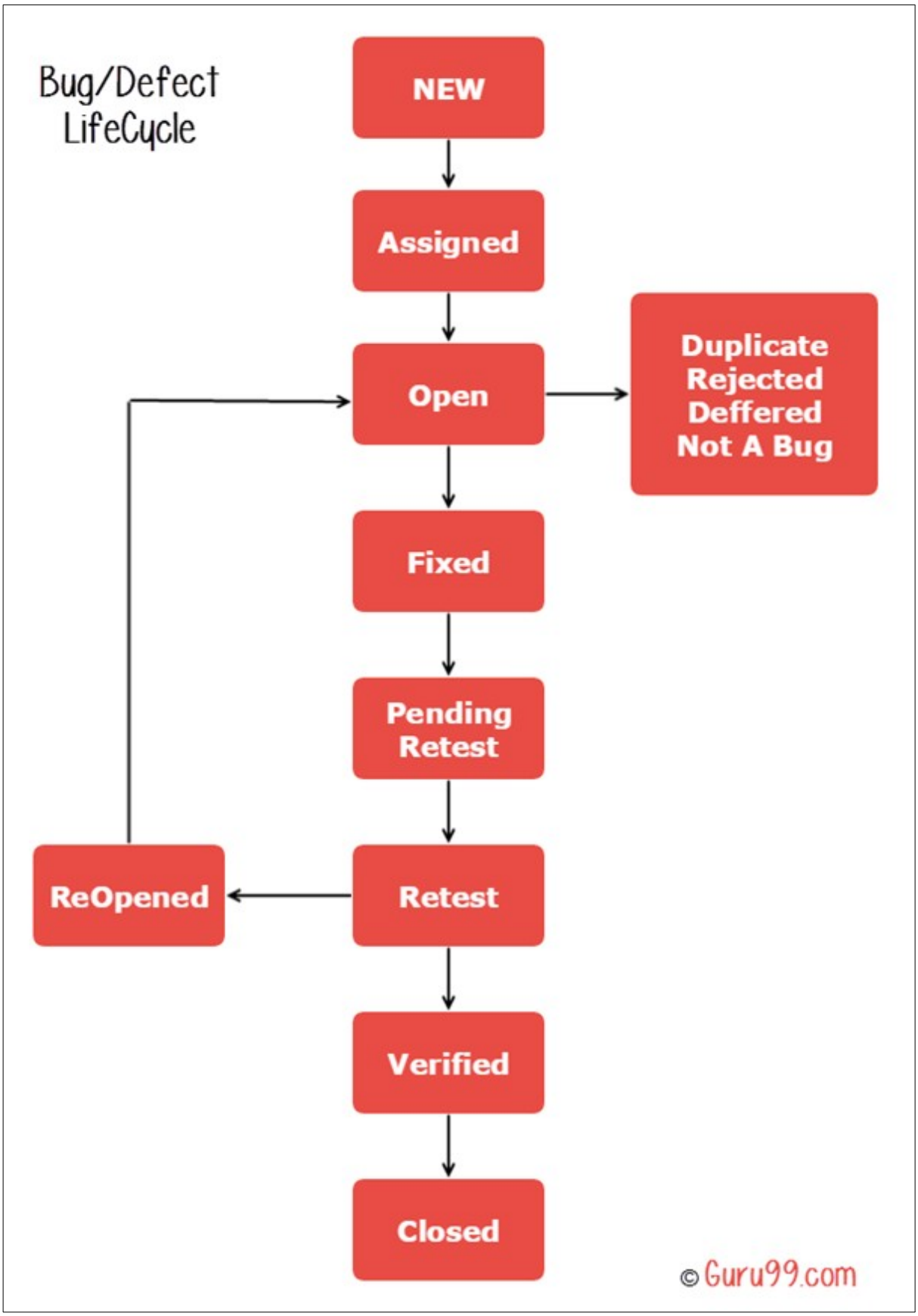
●**Assigned:** Once the bug is posted by the tester, the lead of the tester approves the bug and assigns the bug to the developer team

●**Open**: The developer starts analyzing and works on the defect fix

●**Fixed**: When a developer makes a necessary code change and verifies the change, he or she can make bug status as “Fixed.”

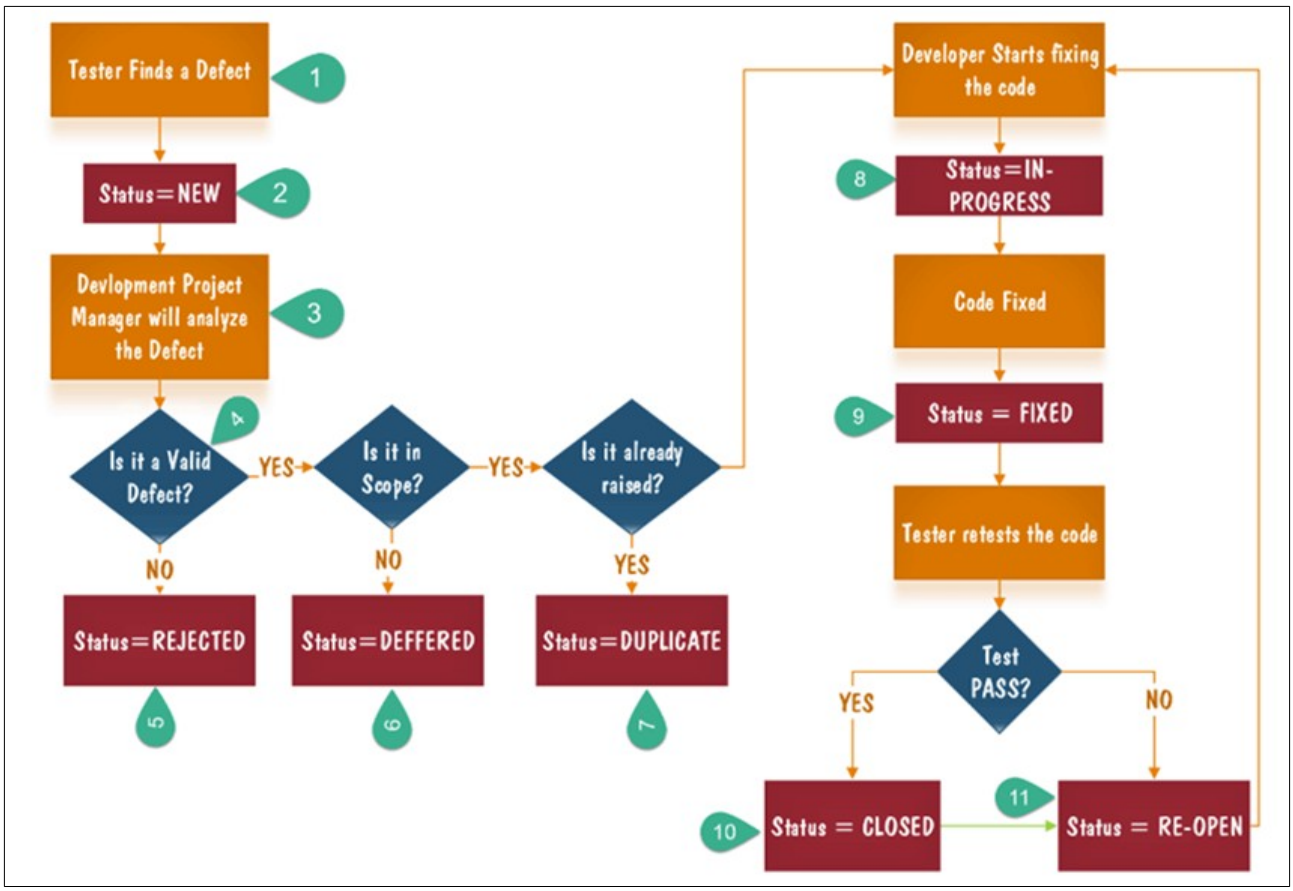
●**Pending retest**: Once the defect is fixed the developer gives a particular code for retesting the code to the tester. Since the software testing remains pending from the testers end, the status assigned is “pending retest.”

●**Retest**: Tester does the retesting of the code at this stage to check whether the defect is fixed by the developer or not and changes the status to “Re-test.”


Bug/Defect LifeCycle
NEW
Assigned
Open
Duplicate
Rejected
Deffered
Not A Bug
Fixed
Pending Retest
ReOpened
Retest
Verified
Closed
©Guru99.com

Verified: The tester re-tests the bug after it got fixed by the developer. If there is no bug detected in the software, then the bug is fixed and the status assigned is “verified.”

- **Reopen**: If the bug persists even after the developer has fixed the bug, the tester changes the status to “reopened”. Once again the bug goes through the life cycle.
- **Closed**: If the bug is no longer exists then tester assigns the status “Closed.”
- **Duplicate**: If the defect is repeated twice or the defect corresponds to the same concept of the bug, the status is changed to “duplicate.”
- **Rejected**: If the developer feels the defect is not a genuine defect then it changes the defect to “rejected.”
- **Deferred**: If the present bug is not of a prime priority and if it is expected to get fixed in the next release, then status “Deferred” is assigned to such bugs
- **Not a bug**: If it does not affect the functionality of the application then the status assigned to a bug is “Not a bug”.



Tester finds the defect

1.Status assigned to defect- New

2.A defect is forwarded to Project Manager for analyze

3.Project Manager decides whether a defect is valid

4.Here the defect is not valid- a status is given “Rejected.”

5.So, project manager assigns a status **rejected**. If the defect is not rejected then the next step is to check whether it is in scope. Suppose we have another function- email functionality for the same application, and you find a problem with that. But it is not a part of the current release when such defects are assigned as a **postponed or deferred** status.

6.Next, the manager verifies whether a similar defect was raised earlier. If yes defect is assigned a status **duplicate**.

7.If no the defect is assigned to the developer who starts fixing the code. During this stage, the defect is assigned a status **in- progress.**

8.Once the code is fixed. A defect is assigned a status **fixed**

9.Next, the tester will re-test the code. In case, the Test Case passes the defect is **closed.** If the test cases fail again, the defect is **re-opened** and assigned to the developer.

10.Consider a situation where during the 1st release of Flight Reservation a defect was found in Fax order that was fixed and assigned a status closed. During the second upgrade release the same defect again re-surfaced. In such cases, a closed defect will be **re-opened.**

That’s all to Bug Life Cycle

This training video describes the various stages in a bug aka defect life cycle and its importance with the help of an example

PART H Software Quality Assurance

Definition of Quality

Software quality is defined as a field of study and practice that describes the desirable attributes of software products. There are two main approaches to software quality: defect management and quality attributes.

SOFTWARE QUALITY DEFECT MANAGEMENT APPROACH

A software defect can be regarded as any failure to address end-user requirements. Common defects include missed or misunderstood requirements and errors in design, functional logic, data relationships, process timing, validity checking, and coding errors.

The software defect management approach is based on counting and managing defects. Defects are commonly categorized by severity, and the numbers in each category are used for planning. More mature software development organizations use tools, such as defect leakage matrices (for counting the numbers of defects that pass through development phases prior to detection) and control charts, to measure and improve development process capability.

SOFTWARE QUALITY ATTRIBUTES APPROACH

This approach to software quality is best exemplified by fixed quality models, such as ISO/IEC 25010:2011. This standard describes a hierarchy of eight quality characteristics, each composed of sub-characteristics:

1.Functional suitability

2.Reliability

3.Operability

4.Performance efficiency

5.Security

6.Compatibility

7.Maintainability

8.Transferability

Testing and Quality Assurance at Workplace

What is Assurance?

Assurance is nothing but a positive declaration on a product or service, which gives confidence. It is certainty of a product or a service, which it will work well. It provides a guarantee that the product will work without any problems as per the expectations or requirements.

Quality Assurance in Software Testing

Quality Assurance in Software Testing is defined as a procedure to ensure the quality of software products or services provided to the customers by an organization. Quality assurance focuses on improving the software development process and making it efficient and effective as per the quality standards defined for software products. Quality Assurance is popularly known as QA Testing.

How to do Quality Assurance: Complete Process

Quality Assurance methodology has a defined cycle called PDCA cycle or Deming cycle. The phases of this cycle are:

- **Plan**
- **Do**
- **Check**
- **Act**

What is Quality Control?

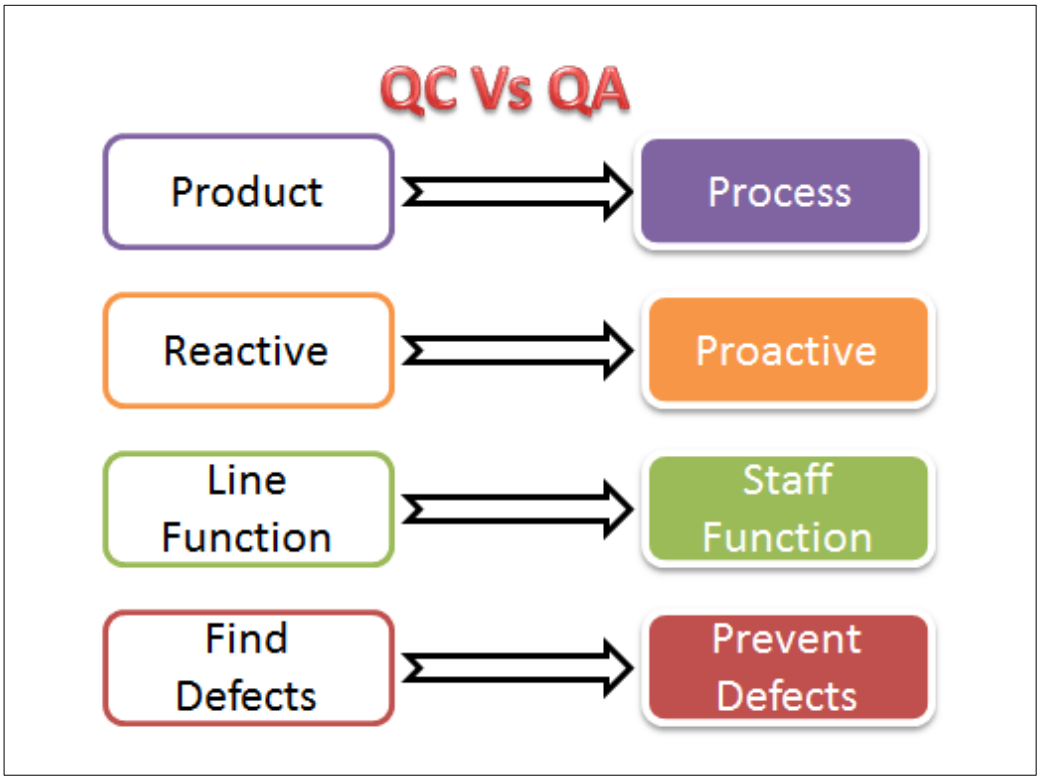
Quality control popularly abbreviated as QC. It is a Software Engineering process used to ensure quality in a product or a service. It does not deal with the processes used to create a product; rather it examines the quality of the “end products” and the final outcome.

The main aim of Quality control is to check whether the products meet the specifications and requirements of the customer. If an issue or problem is identified, it needs to be fixed before delivery to the customer.

QC also evaluates people on their quality level skill sets and imparts training and certifications. This evaluation is required for the service based organization and helps provide “perfect” service to the customers.

Difference between Quality Control and Quality Assurance?

Sometimes, QC is confused with the QA. Quality control is to examine the product or service and check for the result. Quality Assurance in Software Engineering is to examine the processes and make changes to the processes which led to the end-product.



Examples of QC and QA activities are as follows:

Quality Control Activities	Quality Assurance Activities
Walkthrough	Quality Audit
Testing	Defining Process
Inspection	Tool Identification and selection
Checkpoint review	Training of Quality Standards and Processes

The above activities are concerned with Quality Assurance and Control mechanisms for any product and not essentially software. With respect to software

- QA becomes SQA (Software Quality Assurance)
- QC becomes Software Testing.

Also Check:- Quality Assurance Vs Quality Control: What's the Difference?

Differences between SQA and Software Testing

Following table explains on differences between SQA and Software Testing:

SQA	Software Testing
Software Quality Assurance is about engineering process	Software Testing is to test a product for

that ensures quality	problems before the product goes live
Involves activities related to the implementation of processes, procedures, and standards. Example – Audits Training	Involves actives concerning verification of product Example – Review Testing
Process focused	Product focused
Preventive technique	Corrective technique
Proactive measure	Reactive measure
The scope of SQA applied to all products that will be created by the organization	The scope of Software Testing applies to a particular product being tested.

Best practices for Quality Assurance:

- Create a Robust Testing Environment
- Select release criteria carefully
- Apply automated testing to high-risk areas to save money. It helps to fasten the entire process.
- Allocate Time Appropriately for each process
- It is important to prioritize bugs fixes based on software usage
- Form dedicated security and performance testing team
- Simulate customer accounts similar to a production environment

Quality Assurance Functions:

There are 5 primary Quality Assurance Functions:

1.**Technology transfer:** This function involves getting a product design document as well as trial and error data and its evaluation. The documents are distributed, checked and approved

2.**Validation:** Here validation master plan for the entire system is prepared. Approval of test criteria for validating product and process is set. Resource planning for execution of a validation plan is done.

3.**Documentation:** This function controls the distribution and archiving of documents. Any change in a document is made by adopting the proper change control procedure. Approval of all types of documents.

4.**Assuring Quality of products**

5.**Quality improvement plans**

Quality Assurance Certifications:

There are several certifications available in the industry to ensure that Organizations follow Standards Quality Processes. Customers make this as qualifying criteria while selecting a software vendor.

ISO 9000

This standard was first established in 1987, and it is related to Quality Management Systems. This helps the organization ensure quality to their customers and other stakeholders. An organization who wishes to be certified as ISO 9000 is audited based on their functions, products, services and their processes. The main objective is to review and verify whether the organization is following the process as expected and check whether existing processes need improvement.

This certification helps –

- Increase the profit of the organization
- Improves Domestic and International trade
- Reduces waste and increase the productivity of the employees
- Provide Excellent customer satisfaction

➢CMMI level

The **Capability Maturity Model Integrated (CMMI)** is a process improvement approach developed specially for software process improvement. It is based on the process maturity framework and used as a general aid in business processes in the Software Industry. This model is highly regarded and widely used in Software Development Organizations.

CMMI has 5 levels. An organization is certified at CMMI level 1 to 5 based on the maturity of their Quality Assurance Mechanisms.

- Level 1 – **Initial:** In this stage the quality environment is unstable. Simply, no processes have been followed or documented
- Level 2 – **Repeatable:** Some processes are followed which are repeatable. This level ensures processes are followed at the project level.
- Level 3 – **Defined:** Set of processes are defined and documented at the organizational level. Those defined processes are subject to some degree of improvement.
- Level 4 – **Managed:** This level uses process metrics and effectively controls the processes that are followed.
- Level 5 – **Optimizing:** This level focuses on the continuous improvements of the processes through learning & innovation.

Also Check:- Capability Maturity Model (CMM) & it's Levels in Software Engineering

Test Maturity Model (TMM):

This model assesses the maturity of processes in a Testing Environment. Even this model has 5 levels, defined below-

- Level 1 – **Initial**: There is no quality standard followed for testing processes and only ad-hoc methods are used at this level

➢Level 2 – **Definition:** Defined process. Preparation of test strategy, plans, test cases are done.

➢Level 3 – **Integration:** Testing is carried out throughout the software development lifecycle (SDLC) – which is nothing but integration with the development activities, E.g., V-Model.

➢Level 4 – **Management and Measurement:** Review of requirements and designs takes place at this level and criteria has been set for each level of testing

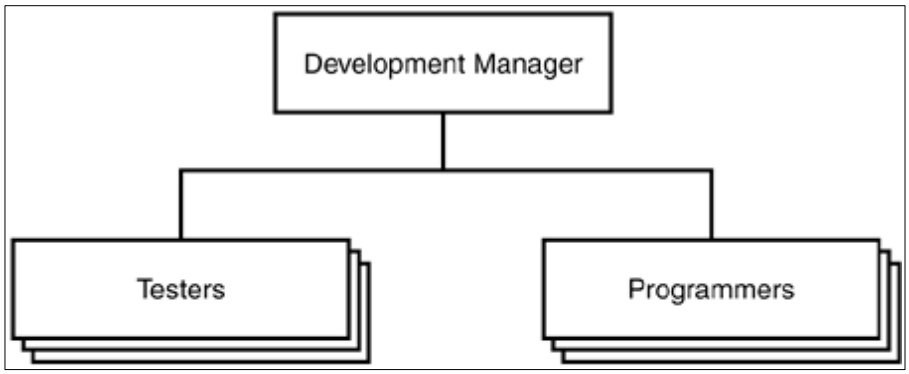
➢Level 5 – **Optimization:** Many preventive techniques are used for testing processes, and tool support(Automation) is used to improve the testing standards and processes.

Also Check:- What is Test Maturity Model (TMM) in Software Testing?

Test Management and Organizational Structure

Besides a test group's name and its assumed responsibilities, there's another attribute that greatly affects what it does and how it works with the project team. That attribute is where it fits in the company's overall management structure. A number of organizational structures are possible, each having its own positives and negatives. Some are claimed to be generally better than others, but what's better for one may not necessarily be better for another. If you work for any length of time in software testing, you'll be exposed to many of them. Here are a few common examples.

Figure 21.2 shows a structure often used by small (fewer than 10 or so people) project teams. In this structure, the test group reports into the Development Manager, the person managing the work of the programmers. Given what you've learned about software testing, this should raise a red flag of warning to youthe people writing the code and the people finding bugs in that code reporting to the same person has the potential for big problem

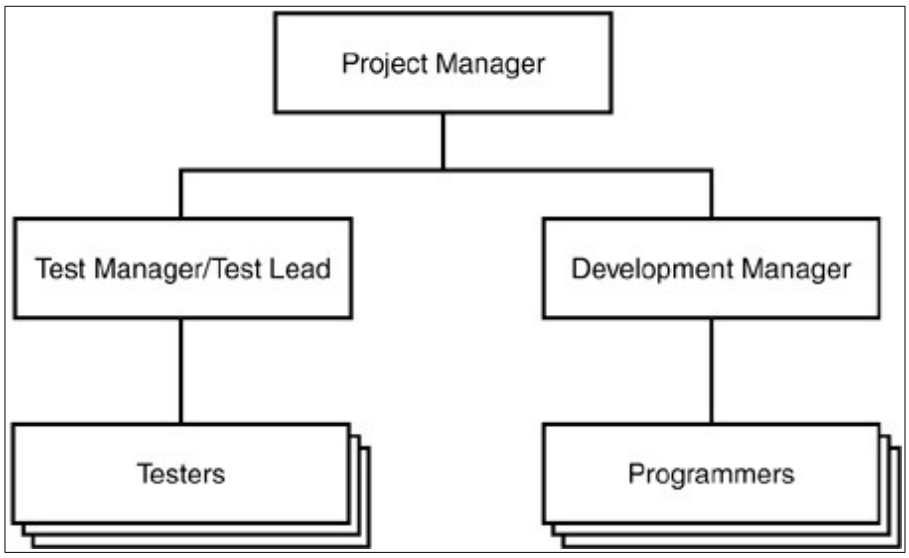


There's the inevitable conflict of interest. The Development Manager's goal is to have his team develop software. Testers reporting bugs just hinder that process. Testers doing their job well on one side make the programmers look bad on the other. If the manager gives more resources and funding to the testers, they'll probably find more bugs, but the more bugs they find, the more they'll crimp the manager's goals of making software.

Despite these negatives, this structure can work well if the development manager is very experienced and realizes that his goal isn't just to create software, but to create quality software. Such a manager would value the testers as equals to the programmers. This is also a very good organization for communications flow. There are minimal layers of management and the testers and programmers can very efficiently work together.

Figure 21.3 shows another common organizational structure where both the test group and the development group report to the manager of the project. In this arrangement, the test group often has its own lead or manager whose interest and attention is focused on the test team and their work. This independence is a great advantage when critical decisions are made regarding the software's quality. The test team's voice is equal to the voices of the programmers and other groups contributing to the product.

Figure 21.3. In an organization where the test team reports to the project manager, there's some independence of the testers from the programmers.



Software Quality Assurance Metrics

Software metrics can be classified into three categories −

- Product metrics − Describes the characteristics of the product such as size, complexity, design features, performance, and quality level.
- Process metrics − These characteristics can be used to improve the development and maintenance activities of the software.
- Project metrics − This metrics describe the project characteristics and execution. Examples include the number of software developers, the staffing pattern over the life cycle of the software, cost, schedule, and productivity.

Some metrics belong to multiple categories. For example, the in-process quality metrics of a project are both process metrics and project metrics.

Software quality metrics are a subset of software metrics that focus on the quality aspects of the product, process, and project. These are more closely associated with process and product metrics than with project metrics.

Software quality metrics can be further divided into three categories −

- Product quality metrics
- In-process quality metrics
- Maintenance quality metrics

Product Quality Metrics

This metrics include the following −

- Mean Time to Failure
- Defect Density
- Customer Problems
- Customer Satisfaction

Mean Time to Failure

It is the time between failures. This metric is mostly used with safety critical systems such as the airline traffic control systems, avionics, and weapons.

Defect Density

It measures the defects relative to the software size expressed as lines of code or function point, etc. i.e., it measures code quality per unit. This metric is used in many commercial software systems.

Customer Problems

It measures the problems that customers encounter when using the product. It contains the customer's perspective towards the problem space of the software, which includes the non-defect oriented problems together with the defect problems.

In-process Quality Metrics

In-process quality metrics deals with the tracking of defect arrival during formal machine testing for some organizations. This metric includes −

- Defect density during machine testing
- Defect arrival pattern during machine testing
- Phase-based defect removal pattern
- Defect removal effectiveness

Defect density during machine testing

Defect rate during formal machine testing (testing after code is integrated into the system library) is correlated with the defect rate in the field. Higher defect rates found during testing is an indicator that the software has experienced higher error injection during its development process, unless the higher testing defect rate is due to an extraordinary testing effort.

This simple metric of defects per KLOC or function point is a good indicator of quality, while the software is still being tested. It is especially useful to monitor subsequent releases of a product in the same development organization.

Defect arrival pattern during machine testing

The overall defect density during testing will provide only the summary of the defects. The pattern of defect arrivals gives more information about different quality levels in the field. It includes the following −

- The defect arrivals or defects reported during the testing phase by time interval (e.g., week). Here all of which will not be valid defects.
- The pattern of valid defect arrivals when problem determination is done on the reported problems. This is the true defect pattern.
- The pattern of defect backlog overtime. This metric is needed because development organizations cannot investigate and fix all the reported problems immediately. This is a workload statement as well as a quality statement. If the defect backlog is large at the end of the development cycle and a lot of fixes have yet to be integrated into the system, the stability of the system (hence its quality) will be affected. Retesting (regression test) is needed to ensure that targeted product quality levels are reached.

Phase-based defect removal pattern

This is an extension of the defect density metric during testing. In addition to testing, it tracks the defects at all phases of the development cycle, including the design reviews, code inspections, and formal verifications before testing.

Because a large percentage of programming defects is related to design problems, conducting formal reviews, or functional verifications to enhance the defect removal capability of the process at the front-end reduces error in the software. The pattern of phase-based defect removal reflects the overall defect removal ability of the development process.

With regard to the metrics for the design and coding phases, in addition to defect rates, many development organizations use metrics such as inspection coverage and inspection effort for in-process quality management.

Defect removal effectiveness

It can be defined as follows −

DRE=DefectremovedduringadevelopmentphaseDefectslatentintheproduct×100%

This metric can be calculated for the entire development process, for the front-end before code integration and for each phase. It is called early defect removal when used for the front-end and phase effectiveness for specific phases. The higher the value of the metric, the more effective the development process and the fewer the defects passed to the next phase or to the field. This metric is a key concept of the defect removal model for software development.

Maintenance Quality Metrics

Although much cannot be done to alter the quality of the product during this phase, following are the fixes that can be carried out to eliminate the defects as soon as possible with excellent fix quality.

- Fix backlog and backlog management index
- Fix response time and fix responsiveness
- Percent delinquent fixes
- Fix quality

Fix backlog and backlog management index

Fix backlog is related to the rate of defect arrivals and the rate at which fixes for reported problems become available. It is a simple count of reported problems that remain at the end of each month or each week. Using it in the format of a trend chart, this metric can provide meaningful information for managing the maintenance process.

Backlog Management Index (BMI) is used to manage the backlog of open and unresolved problems.

BMI=NumberofproblemsclosedduringthemonthNumberofproblemsarrivedduringthemonth×100%

If BMI is larger than 100, it means the backlog is reduced. If BMI is less than 100, then the backlog increased.

Fix response time and fix responsiveness

The fix response time metric is usually calculated as the mean time of all problems from open to close. Short fix response time leads to customer satisfaction.

The important elements of fix responsiveness are customer expectations, the agreed-to fix time, and the ability to meet one's commitment to the customer.

Percent delinquent fixes

It is calculated as follows −

PercentDelinquentFixes=

Numberoffixesthatexceededtheresponsetimecriteriabyceveritylevel Numberoffixesdeliveredinaspecifiedtime×100%

Fix Quality

Fix quality or the number of defective fixes is another important quality metric for the maintenance phase. A fix is defective if it did not fix the reported problem, or if it fixed the original problem but injected a new defect. For mission-critical software, defective fixes are detrimental to customer satisfaction. The metric of percent defective fixes is the percentage of all fixes in a time interval that is defective.

A defective fix can be recorded in two ways: Record it in the month it was discovered or record it in the month the fix was delivered. The first is a customer measure; the second is a process measure. The difference between the two dates is the latent period of the defective fix.

Usually the longer the latency, the more will be the customers that get affected. If the number of defects is large, then the small value of the percentage metric will show an optimistic picture. The quality goal for the maintenance process, of course, is zero defective fixes without delinquency.

Quality Management in IT.

Most of the organizations introduced quality management systems many years ago usually based on ISO/IEC 9001. Some organizations even introduced service management system either driven by ITIL processes or service management norms like ISO/IEC 20000. Some organization are structured per COBIT enabling processes, like APO11 Manage Quality. Those norms and frameworks are in great help to demonstrate external customers focus on quality and structured approach to service quality deviations impacting customers

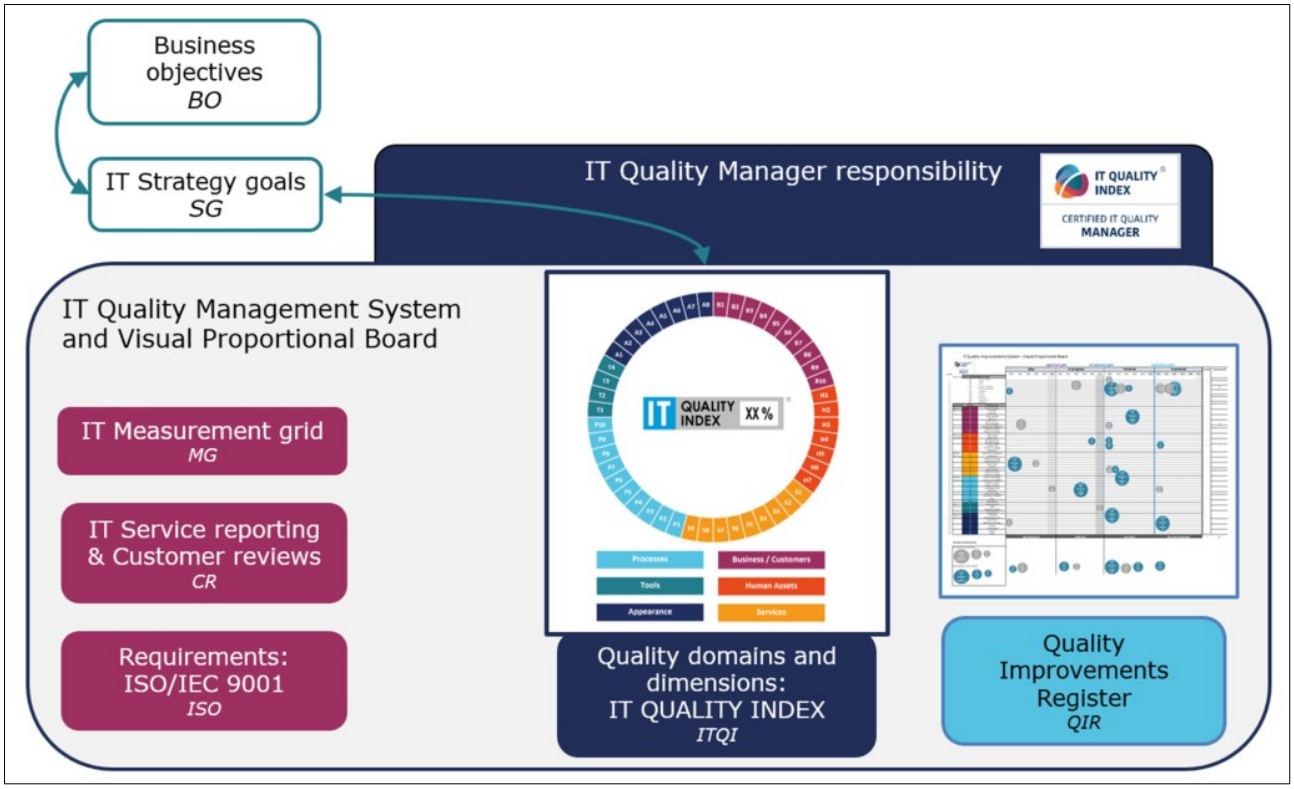
Structured IT Quality Management System needs to be introduced in all IT departments as self-improvement is the only way to commit proactive IT management rather than reactive focused on incidents, problems and requests. The new role of IT quality manager providing answers to all question put in the beginning is needed. IT QMS with visual support, right tool for tracking improvements in IT Quality Index structure, right organization design needs to be introduced.

Value to business:

- •Proactive IT can demonstrate the value of improvements and costs of bad quality
- •Changing mindset and communication style either in IT and all IT/Organization interactions
- •Encourage faster innovations of practices in IT
- •The constant search for better ways of doing all IT activities

IT Quality Management System – key parts:

- •Design of IT QMS based on standard templates adapted to customer context
- •Mapping and interconnection with organization quality management system
- •Role of IT Quality Manager
- •Tool support for improvement tracking, visual proportional board
- •IT Quality Board and IT quality reviews
- •Valuation of improvements technique – CVC, RBA
- •Mapping to COBIT 2019 enabling processes, APO11 Managed Quality. Mapping to SFIA skills – QUAM, Quality management
- •Proactive reporting and visualization


Business objectives
BO
IT Strategy goals
SG
IT Quality Manager responsibility
IT QUALITY INDEX
CERTIFIED IT QUALITY MANAGER
IT Quality Management System and Visual Proportional Board
IT Measurement grid
MG
IT Service reporting & Customer reviews
CR
Requirements: ISO/IEC 9001
ISO
IT QUALITY INDEX XX %
Processes
Business / Customers
Tools
Human Assets
Appearance
Services
Quality domains and dimensions: IT QUALITY INDEX
ITQI
Quality Improvements Register
QIR

PART I Organizational Structure

CMM (Capability Maturity Model)

CMM was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987.

•It is not a software process model. It is a framework that is used to analyze the approach and techniques followed by any organization to develop software products.

•It also provides guidelines to further enhance the maturity of the process used to develop those software products.

•It is based on profound feedback and development practices adopted by the most successful organizations worldwide.

•This model describes a strategy for software process improvement that should be followed by moving through 5 different levels.

•Each level of maturity shows a process capability level. All the levels except level-1 are further described by Key Process Areas (KPA's).

Shortcomings of SEI/CMM:

•It encourages the achievement of a higher maturity level in some cases by displacing the true mission, which is improving the process and overall software quality.

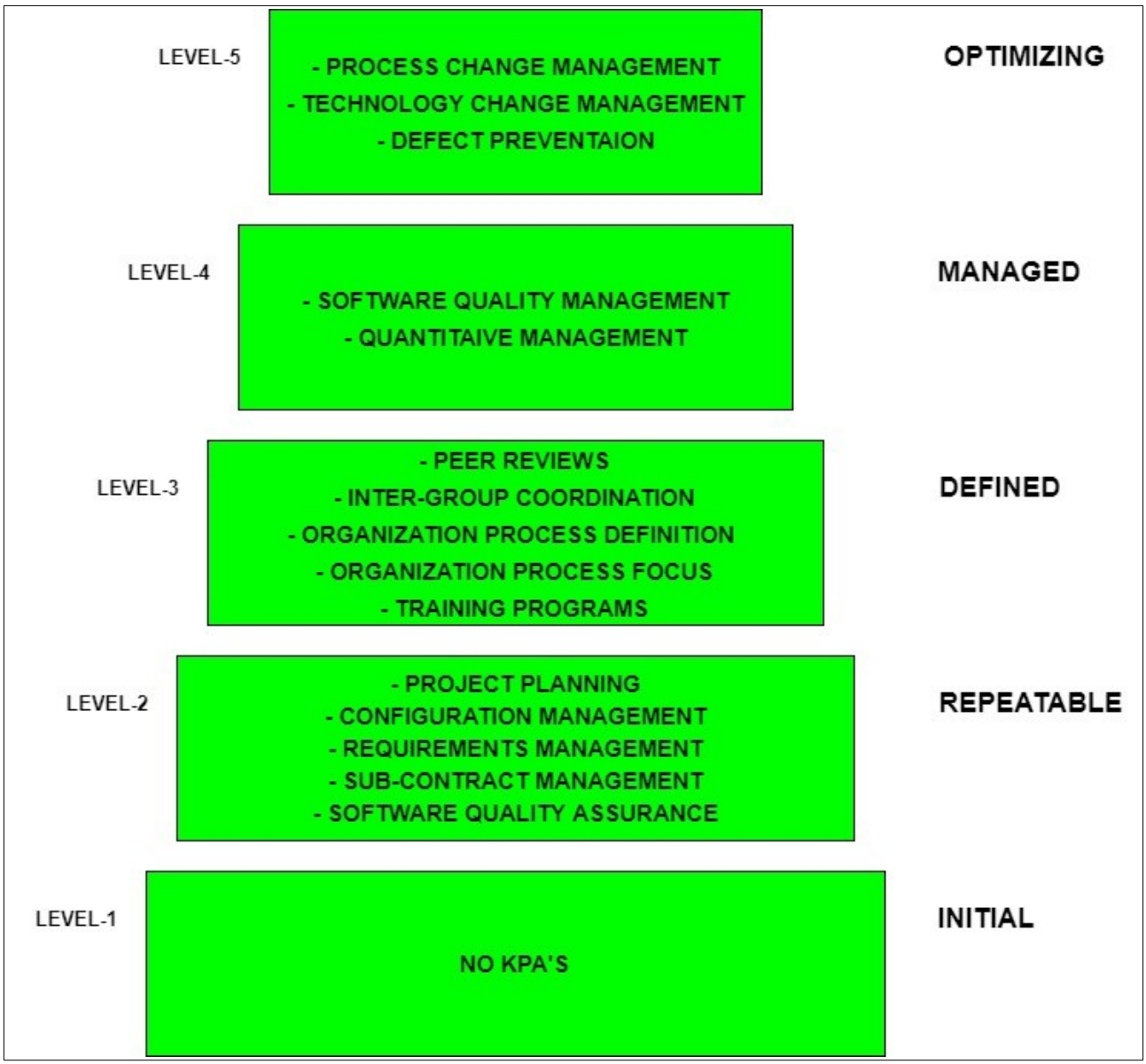
•It only helps if it is put into place early in the software development process.

•It has no formal theoretical basis and in fact is based on the experience of very knowledgeable people.

•It does not have good empirical support and this same empirical support could also be constructed to support other models.

Key Process Areas (KPA's):
Each of these KPA's defines the basic requirements that should be met by a software process in order to satisfy the KPA and achieve that level of maturity.

Conceptually, key process areas form the basis for management control of the software project and establish a context in which technical methods are applied, work products like models, documents, data, reports, etc. are produced, milestones are established, quality is ensured and change is properly managed.



Level-1: Initial –

•No KPA's defined.

•Processes followed are Adhoc and immature and are not well defined.

•Unstable environment for software development.

•No basis for predicting product quality, time for completion, etc.

Level-2: Repeatable –

•Focuses on establishing basic project management policies.

•Experience with earlier projects is used for managing new similar natured projects.

•**Project Planning-** It includes defining resources required, goals, constraints, etc. for the project. It presents a detailed plan to be followed systematically for the successful completion of good quality software.

•**Configuration Management-** The focus is on maintaining the performance of the software product, including all its components, for the entire lifecycle.

•**Requirements Management-** It includes the management of customer reviews and feedback which result in some changes in the requirement set. It also consists of accommodation of those modified requirements.

•**Subcontract Management-** It focuses on the effective management of qualified software contractors i.e. it manages the parts of the software which are developed by third parties.

•**Software Quality Assurance-** It guarantees a good quality software product by following certain rules and quality standard guidelines while developing.

Level-3: Defined –

•At this level, documentation of the standard guidelines and procedures takes place.

•It is a well-defined integrated set of project-specific software engineering and management processes.

•**Peer Reviews-** In this method, defects are removed by using a number of review methods like walkthroughs, inspections, buddy checks, etc.

•**Intergroup Coordination-** It consists of planned interactions between different development teams to ensure efficient and proper fulfillment of customer needs.

•**Organization Process Definition-** Its key focus is on the development and maintenance of the standard development processes.

•**Organization Process Focus-** It includes activities and practices that should be followed to improve the process capabilities of an organization.

•**Training Programs-** It focuses on the enhancement of knowledge and skills of the team members including the developers and ensuring an increase in work efficiency.

Level-4: Managed –

•At this stage, quantitative quality goals are set for the organization for software products as well as software processes.

•The measurements made help the organization to predict the product and process quality within some limits defined quantitatively.

•**Software Quality Management-** It includes the establishment of plans and strategies to develop quantitative analysis and understanding of the product’s quality.

•**Quantitative Management-** It focuses on controlling the project performance in a quantitative manner.

Level-5: Optimizing –

•This is the highest level of process maturity in CMM and focuses on continuous process improvement in the organization using quantitative feedback.

•Use of new tools, techniques, and evaluation of software processes is done to prevent recurrence of known defects.

•**Process Change Management-** Its focus is on the continuous improvement of the organization's software processes to improve productivity, quality, and cycle time for the software product.

•**Technology Change Management-** It consists of the identification and use of new technologies to improve product quality and decrease product development time.

•**Defect Prevention-** It focuses on the identification of causes of defects and prevents them from recurring in future projects by improving project-defined processes.

ISO 9000

ISO 9000 is defined as a set of international standards on quality management and quality assurance developed to help companies effectively document the quality system elements needed to maintain an efficient quality system. They are not specific to any one industry and can be applied to organizations of any size.

ISO 9000 can help a company satisfy its customers, meet regulatory requirements, and achieve continual improvement. It should be considered to be a first step or the base level of a quality system.

ISO 9000:2015 principles of Quality Management

The ISO 9000:2015 and ISO 9001:2015 standards are based on seven quality management principles that senior management can apply to promote organizational improvement.



1. Customer focus

- Understand the needs of existing and future customers
- Align organizational objectives with customer needs and expectations
- Meet customer requirements
- Measure customer satisfaction
- Manage customer relationships
- Aim to exceed customer expectations
- Learn more about the customer experience and customer satisfaction

2.Leadership

- Establish a vision and direction for the organization
- Set challenging goals
- Model organizational values
- Establish trust
- Equip and empower employees
- Recognize employee contributions
- Learn more about leadership

3.Engagement of people

- Ensure that people's abilities are used and valued
- Make people accountable
- Enable participation in continual improvement
- Evaluate individual performance
- Enable learning and knowledge sharing
- Enable open discussion of problems and constraints
- Learn more about employee involvement

4.Process approach

- Manage activities as processes
- Measure the capability of activities
- Identify linkages between activities

- Prioritize improvement opportunities
- Deploy resources effectively
- Learn more about a process view of work and see process analysis tools

5.Improvement

- Improve organizational performance and capabilities
- Align improvement activities
- Empower people to make improvements
- Measure improvement consistently
- Celebrate improvements
- Learn more about approaches to continual improvement

6.Evidence-based decision making

- Ensure the accessibility of accurate and reliable data
- Use appropriate methods to analyze data
- Make decisions based on analysis
- Balance data analysis with practical experience
- See tools for decision making

7.Relationship management

- Identify and select suppliers to manage costs, optimize resources, and create value
- Establish relationships considering both the short and long term
- Share expertise, resources, information, and plans with partners
- Collaborate on improvement and development activities
- Recognize supplier successes
- Learn more about supplier quality and see resources related to managing the supply chain

Software Engineering Standards

A software engineering standard may be defined as any standard, protocol, or similar document that outlines the rules and processes for the creation of software products. A typical software development company will have such documents on hand, and they will be intended for usage only by that specific organization.

ISO standards for software development

ISO and IEEE are two of the most well-known international standards that enable software businesses all around the globe better structure their operations by providing a well-defined and effective framework.

ISO - International Organization for Standardization

IEEE - Electrical and Electronics Engineers

These standards are a list of principles and good practices that are used to help software companies increase the quality of their services and organisation of processes to reach better results. The ISO is a global standard-setting organization that spans sectors including software and food. ISO's major purpose is to assist businesses to improve their processes and therefore their services by offering a set of standards and norms to follow:

- **ISO/IEC 12207 and ISO/IEC 15288** standards are the most important for software development and can be replaced by each other, both referring to the Software life cycle processes. Other standards that are relevant to software development:

- **ISO/IEC 15939 (Software measurement process)** specifies the activities and tasks that are required to create, implement, and enhance the software measurement process. It doesn't provide a list of software metrics or give tools for estimating the qualities of a software product or process.

- **ISO/IEC 14143 (Software measurement - Functional size measurement).** Although these standards are used to assist software development, they are unrelated to the current issue and will not be discussed further.

ISO 12207 Software life cycle processes

ISO/IEC 12207 describes the following parts as:

- **Software product -** "The set of computer programs, procedures, and possibly associated documentation and data."

- **Software service -** "Performance of activities, work, or duties connected with a software product, such as its development, maintenance, and operation."

- **System -** "An integrated composite that consists of one or more of the processes, hardware, software, facilities, and people, that provides a capability to satisfy a stated need or objective."

ISO 29119: Software Testing

The ISO 29119 set of standards outlines best practices for software and systems engineering, including software testing. These best practices aren't tied to any particular development model, but many of them resemble a V-model approach.

For software testing, ISO 29119 is divided into five sections:

- Testing based on keywords
- Documentation for testing
- Techniques and techniques for testing
- Definitions and concepts

ISO 27001: Information Security

The ISO 2700 family, which encompasses information security requirements inside an enterprise, is another prominent standard among software development organizations. ISO 2700's major purpose is to secure a company's assets and enhance its security procedures.

The most significant advantage of an ISO 27001 accreditation for a firm is the assurance of the security of its operations and data. In this approach, a firm seems more dependable and trustworthy to customers while maintaining a competitive edge over other enterprises.

The following are some of the most important ISO 27001 clauses:

- Management of information security
- System design for information security management
- Risk assessment and management
- A performance evaluation of the system
- Developing and implementing a corrective action strategy